BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CMC

KHOA: VI ĐIỆN TỬ - VIỄN THÔNG

CMC UNIVERSITY

Aspire to Inspire the Digital World

CÔNG TRÌNH THAM DỰ

CUỘC THI SINH VIÊN NGHIÊN CỨU KHOA HỌC

NĂM HỌC 2025 – 2026

Tên công trình: Mạng cảm biến IoT giám sát và cảnh báo chất lượng không khí học đường.

Nhóm sinh viên thực hiện:

1. Họ và tên: Phạm Hải Đăng, mã SV: BEC240010, lớp 24EC, khóa K3
2. Họ và tên: Cao Hoàng Minh, mã SV: BEC240035, lớp 24EC, khóa K3
3. Họ và tên: Nguyễn Trí Đạt, mã SV: BEC240013, lớp: 24EC, khóa: K3
4. Họ và tên: Lê Dương Tùng Anh, mã SV: BEC240004, lớp: 24EC, khóa: K3
5. Họ và tên: Nguyễn Hà Nam Khánh, mã SV: BEC240026, lớp: 24EC, khóa: K3

Đơn vị: Khoa vi điện tử - Viễn thông

Người hướng dẫn: TS. Nguyễn Thị Thu Hằng

Hà Nội, 05/2026

TÓM TẮT CÔNG TRÌNH

Đề tài tập trung nghiên cứu, thiết kế và chế tạo một hệ thống Internet of Things (IoT) giám sát chất lượng không khí cục bộ, hướng tới giải quyết bài toán quản lý vi khí hậu trong môi trường trường học. Về mặt phần cứng, hệ thống sử dụng vi điều khiển ESP32 làm thiết bị thu thập dữ liệu trung tâm, tích hợp cùng cụm cảm biến đa lượng bao gồm: DHT11 (nhiệt độ, độ ẩm), SGP30 (nồng độ eCO2 và TVOC) và MQ135 (nồng độ khí gas).

Về mặt phần mềm, toàn bộ dữ liệu môi trường được truyền tải theo thời gian thực qua giao thức MQTT về máy chủ cục bộ (Local Server) được triển khai trên nền tảng mã nguồn mở Home Assistant (HA). Điểm nhấn kỹ thuật của nghiên cứu là việc xây dựng thuật toán đánh giá rủi ro (Scoring Algorithm) bằng ngôn ngữ Jinja2, cho phép phân tích chéo các thông số đầu vào để đưa ra cấp độ cảnh báo. Kết quả thực nghiệm cho thấy hệ thống đáp ứng tốt khả năng giám sát liên tục với độ trễ thấp; tự động thay đổi giao diện cảnh báo trực quan trên bảng điều khiển (Dashboard) và đồng bộ thông báo đẩy (Push Notifications) đến thiết bị di động của người dùng khi phát hiện nồng độ khí độc vượt ngưỡng định mức.

LỜI CAM ĐOAN

Chúng em xin cam đoan đây là công trình nghiên cứu khoa học do chính chúng em thực hiện dưới sự hướng dẫn trực tiếp của giảng viên. Các số liệu, kết quả đo lường và thực nghiệm trình bày trong báo cáo là hoàn toàn trung thực, khách quan và chưa từng được công bố trong bất kỳ công trình nào khác. Mọi thông tin tham khảo, trích dẫn tài liệu đều được ghi rõ nguồn gốc theo đúng quy định.

Nếu phát hiện có sự sao chép hoặc vi phạm bản quyền, chúng em xin chịu hoàn toàn trách nhiệm trước Hội đồng Khoa học và Nhà trường.

Hà Nội, ngày ... tháng ... năm 2026

CÁN BỘ HƯỚNG DẪN **NHÓM SINH VIÊN THỰC HIỆN**

MỤC LỤC

MỤC LỤC **1**

DANH MỤC HÌNH ẢNH **3**

PHẦN 1. TỔNG QUAN VỀ HỆ THỐNG IOT GIÁM SÁT CHẤT LƯỢNG KHÔNG KHÍ **4**

1.1. Lý do chọn đề tài: **4**

1.2. Tổng quan tình hình nghiên cứu trong và ngoài nước: **4**

1.3. Các hệ thống IoT giám sát môi trường hiện nay: **4**

1.4. Định hướng nghiên cứu của đề tài: **5**

PHẦN 2. MỤC TIÊU, NỘI DUNG, PHƯƠNG PHÁP NGHIÊN CỨU, ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU **6**

2.1. Mục tiêu: **6**

2.2. Nội dung nghiên cứu: **6**

2.3. Phương pháp nghiên cứu: **6**

2.4. Đối tượng và phạm vi nghiên cứu: **7**

PHẦN 3. THIẾT KẾ HỆ THỐNG GIÁM SÁT CHẤT LƯỢNG KHÔNG KHÍ **8**

3.1. Lựa chọn và nguyên lý hoạt động của cảm biến: **9**

3.1.1. Cảm biến khí MQ135: 9

3.1.2. Cảm biến nhiệt độ và độ ẩm DHT11: 10

3.1.3. Cảm biến VOC SGP30: 11

3.2. Lựa chọn vi điều khiển: **12**

3.2.1. Tổng quan các nền tảng: 12

3.2.2. Lý do chọn ESP32: 13

3.2.3. Đa dạng hóa giao tiếp và khả năng mở rộng hệ thống: 13

PHẦN 4. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG DEMO **15**

4.1. Thiết kế phần cứng hệ thống: **15**

4.2. Xây dựng phần mềm nhúng (Firmware): **15**

4.3. Truyền và lưu trữ dữ liệu (InfluxDB): **16**

4.4. Xây dựng dashboard giám sát (Grafana): **16**

4.5. Luồng hoạt động của hệ thống: **16**

PHẦN 5. CÁCH LẮP MẠCH, THIẾT LẬP, HIỂN THỊ CÁC CHỈ SỐ TRÊN HOME ASSISTANT VÀ THÔNG BÁO RA MÀN HÌNH ĐIỆN THOẠI 17

5.1. Lắp ráp và đấu nối sơ đồ phần cứng: 17

5.2. Lập trình và nạp mã nguồn (Firmware) cho ESP32 18

5.3. Cài đặt và cấu hình máy chủ cục bộ Home Assistant: 18

5.4. Tùy biến và thiết kế giao diện hiển thị (Dashboard UI) thông qua HACS: 20

5.5. Thiết lập tự động hóa (Automations) và hệ thống cảnh báo đa cấp độ 21

PHẦN 6. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ 23

6.1. Đo nhiệt độ và độ ẩm: 23

6.1.1. Kết quả nhiệt độ (Giai đoạn 1) : 23

6.1.2. Hoàn thiện và Tích hợp hệ thống tổng thể (Giai đoạn 2):.....24

6.2. Kết quả đo độ ẩm (Sử dụng cảm biến DHT11): 25

6.3. Kết quả giám sát nồng độ eCO2 và TVOC (Sử dụng cảm biến SGP30): .26

6.4. Kết quả đo MQ135: 28

6.5. Hiển thị kết quả qua app Home assistant: 30

TÀI LIỆU THAM KHẢO.....33

PHỤ LỤC 35

DANH MỤC HÌNH ẢNH

HÌNH 1: MQ135 9
HÌNH 2: DHT11 10
HÌNH 3: SGP30 11
HÌNH 4: BUZZER MM-FMD 13
HÌNH 5: NGUYÊN MẪU PHẦN CỨNG TRẠM ĐO KHÔNG KHÍ IOT ĐƯỢC LẮP RÁP TRÊN BREADBOARD 17
HÌNH 6: GIAO DIỆN NẠP MÃ NGUỒN VÀ KIỂM THỬ DỮ LIỆU THÔ TRÊN SERIAL MONITOR 18
HÌNH 7: CẤU HÌNH MÁY CHỦ ẢO VIRTUALBOX 19
HÌNH 8: GIAO DIỆN KHỞI ĐỘNG HOME ASSISTANT CLI 19
HÌNH 9: QUẢN LÝ CÁC GÓI GIAO DIỆN MỞ RỘNG THÔNG QUA THƯ VIỆN HACS 20
HÌNH 10: QUẢN LÝ CÁC KỊCH BẢN TỰ ĐỘNG HÓA TRÊN HOME ASSISTANT 21
HÌNH 11: CẤU TRÚC TRIGGER ĐA LUỒNG XỬ LÝ CẢNH BÁO NGUY HIỂM 21
HÌNH 12: CẤU TRÚC LOGIC ĐỘ TRỄ VÀ KIỂM TRA CHÉO TRONG KỊCH BẢN XÁC NHẬN AN TOÀN 22
HÌNH 13: HÌNH ẢNH MẠCH DEMO ĐO NHIỆT ĐỘ 23
HÌNH 14: MẠCH HIỂN THỊ CẢNH BÁO KHI MÔI TRƯỜNG NÓNG 24
HÌNH 15: MẠCH HIỂN THỊ NHIỆT ĐỘ KHI MÔI TRƯỜNG BÌNH THƯỜNG 24
HÌNH 16: DỮ LIỆU NHIỆT ĐỘ QUA HOME ASSISTANT VÀ ARDUINO IDE 24
HÌNH 17: DỮ LIỆU ĐO ĐỘ ẨM QUA HOME ASSISTANT VÀ ARDUINO IDE 25
HÌNH 18: DỮ LIỆU CO2 QUA HOME ASSISTANT VÀ ARDUINO IDE 26
HÌNH 19: DỮ LIỆU TVOC QUA HOME ASSISTANT VÀ ARDUINO IDE 27
HÌNH 20: DỮ LIỆU KHÍ GAS QUA HOME ASSISTANT VÀ ARDUINO IDE 29
HÌNH 21: GIAO DIỆN GIÁM SÁT CHẤT LƯỢNG KHÔNG KHÍ TỔNG THỂ TRÊN HOME ASSISTANT 30
HÌNH 22: GIAO DIỆN CẢNH BÁO ĐỎ KHI ECO2 VÀ TVOC VƯỢT NGƯỠNG 31
HÌNH 23: CHUỖI THÔNG BÁO ĐẨY HIỂN THỊ TRÊN MÀN HÌNH KHÓA ĐIỆN THOẠI DI ĐỘNG 32

PHẦN 1. TỔNG QUAN VỀ HỆ THỐNG IOT GIÁM SÁT CHẤT LƯỢNG KHÔNG KHÍ

1.1. Lý do chọn đề tài:

Trong bối cảnh ô nhiễm không khí đô thị ngày càng nghiêm trọng, đặc biệt tại Hà Nội – nơi chỉ số PM2.5 thường xuyên vượt ngưỡng an toàn, chất lượng không khí trong lớp học và khu ký túc xá cũng bị ảnh hưởng đáng kể.

Các nghiên cứu cho thấy khi nồng độ CO_2 vượt 1000 ppm hoặc PM2.5 vượt 35 $\mu g/m^3$ sẽ làm giảm khả năng tập trung và ảnh hưởng đến sức khỏe. Tuy nhiên, hiện nay đa số trường học chưa có hệ thống giám sát chi tiết theo từng phòng.

Vì vậy, việc xây dựng một hệ thống IoT chi phí thấp, dễ triển khai để giám sát và cảnh báo chất lượng không khí là cần thiết.

1.2. Tổng quan tình hình nghiên cứu trong và ngoài nước:

Trên thế giới: Việc ứng dụng Internet of Things (IoT) và mạng cảm biến không dây trong giám sát chất lượng không khí trong nhà (IAQ) đã được khẳng định tính hiệu quả qua nhiều nghiên cứu tổng quan [6]. Đặc biệt, môi trường lớp học khép kín đang là điểm nóng nghiên cứu. Nhiều giải pháp tiên tiến đã được đề xuất như nền tảng SchoolAIR [11] hay ứng dụng thuật toán Học máy (Machine Learning) để tăng độ chính xác cho cảm biến giá rẻ [2]. Dù vậy, các hệ thống này thường yêu cầu hạ tầng đám mây (Cloud) phức tạp hoặc chưa chú trọng vào cơ chế quản lý, cảnh báo tức thời tại chỗ (Local).

Tại Việt Nam: Các mạng lưới quan trắc phổ biến hiện nay phần lớn chỉ cung cấp dữ liệu vĩ mô ngoài trời [4]. Gần đây, nhiều nghiên cứu trong nước đã bắt đầu thiết kế các trạm đo vi khí hậu cỡ nhỏ, nhưng đa số mới dừng ở mức thu thập và hiển thị thông số cơ bản [5], thiếu khả năng tích hợp cảnh báo đa cấp độ và phân tích chéo dữ liệu môi trường.

1.3. Các hệ thống IoT giám sát môi trường hiện nay:

Sự phát triển của công nghệ IoT đã tạo điều kiện thuận lợi cho việc xây dựng các hệ thống giám sát môi trường thông minh. Các hệ thống này thường bao gồm ba thành phần chính: cảm biến, bộ xử lý trung tâm và nền tảng lưu trữ – hiển thị dữ liệu.

Các cảm biến phổ biến được sử dụng gồm:

- Cảm biến bụi mịn (PMS5003, SDS011)
- Cảm biến CO_2 (MH-Z19, SCD30)
- Cảm biến nhiệt độ, độ ẩm (SHT31, DHT22)
- Cảm biến khí VOC (SGP30, CCS811)

Vi điều khiển như ESP32 hoặc Arduino đóng vai trò thu thập và truyền dữ liệu qua các giao thức như MQTT hoặc HTTP. Dữ liệu sau đó được lưu trữ trên các nền tảng như InfluxDB và hiển thị thông qua Grafana hoặc các dashboard web.

Ưu điểm của các hệ thống IoT là khả năng giám sát liên tục, chi phí thấp, dễ mở rộng và tích hợp nhiều chức năng cảnh báo. Tuy nhiên, hạn chế chính là độ chính xác của cảm biến giá rẻ và yêu cầu hiệu chuẩn định kỳ.

1.4. Định hướng nghiên cứu của đề tài:

Từ những phân tích trên, có thể thấy rằng vẫn còn thiếu một giải pháp giám sát chất lượng không khí vi mô trong môi trường học đường tại Việt Nam. Do đó, đề tài này hướng tới việc xây dựng một hệ thống IoT đáp ứng các yêu cầu sau:

- Giám sát đa thông số (PM2.5, CO_2, nhiệt độ, độ ẩm, TVOC) theo thời gian thực
- Triển khai ở quy mô lớp học và ký túc xá với chi phí thấp
- Tích hợp hệ thống cảnh báo trực quan và từ xa (đèn, âm thanh, Telegram, Email)
- Đảm bảo độ chính xác thông qua hiệu chuẩn với thiết bị tham chiếu
- Dễ dàng mở rộng thành mạng lưới giám sát trong toàn bộ khuôn viên trường

Đề tài không chỉ mang ý nghĩa nghiên cứu mà còn có tính ứng dụng thực tiễn cao, góp phần cải thiện môi trường học tập và nâng cao nhận thức về chất lượng không khí trong cộng đồng học đường.

PHẦN 2. MỤC TIÊU, NỘI DUNG, PHƯƠNG PHÁP NGHIÊN CỨU, ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

2.1. Mục tiêu:

- **Mục tiêu tổng quát:** Xây dựng mô hình hệ thống IoT giám sát chất lượng không khí trong lớp học, ký túc xá và khuôn viên trường học, có khả năng thu thập dữ liệu theo thời gian thực, phân tích và cảnh báo tự động khi vượt ngưỡng an toàn.
- **Mục tiêu cụ thể:**
  + Thiết kế – lắp ráp thiết bị cảm biến không khí đo CO_2, TVOC, nhiệt độ và độ ẩm
  + Xây dựng dashboard trực quan hiển thị bản đồ chỉ số không khí trong khuôn viên trường
  + Đề xuất mô hình cảnh báo thông minh (qua đèn, âm thanh hoặc Telegram/Email)
  + Thử nghiệm thực tế tại các lớp học hoặc khu KTX tại Hà Nội, đánh giá độ chính xác (MAE/RMSE)

2.2. Nội dung nghiên cứu:

- Khảo sát thực trạng chất lượng không khí trong lớp học/ký túc xá (CO_2, nhiệt độ, TVOC)
- Thiết kế hệ thống IoT gồm cảm biến, vi điều khiển ESP32, kết nối Wi-Fi và cơ sở dữ liệu đám mây
- Lập trình firmware thu thập – hiệu chỉnh – gửi dữ liệu lên máy chủ
- Xây dựng dashboard hiển thị biểu đồ thời gian thực và màu cảnh báo
- Thử nghiệm và đánh giá độ chính xác so với thiết bị tham chiếu

2.3. Phương pháp nghiên cứu:

- Sử dụng cảm biến DHT11, MQ135, SGP30, ,... kết hợp ESP32
- Truyền dữ liệu qua Wi-Fi (MQTT/HTTP) lên InfluxDB và hiển thị bằng Grafana
- Phân tích dữ liệu bằng Arduino
- So sánh trước và sau hiệu chỉnh, tổng hợp và đề xuất mô hình mở rộng

2.4. Đối tượng và phạm vi nghiên cứu:

- **Đối tượng nghiên cứu:**
  - + Chất lượng không khí trong lớp học, ký túc xá, trọ, nhà riêng
  - + Các thông số: nhiệt độ, độ ẩm, TVOC, CO_2, khí gas.
- **Phạm vi nghiên cứu:**
  - + Khu vực: Hà Nội
  - + Quy mô: lớp học, phòng ở sinh viên
  - + Thiết bị: hệ thống IoT sử dụng ESP32

PHẦN 3. THIẾT KẾ HỆ THỐNG GIÁM SÁT CHẤT LƯỢNG KHÔNG KHÍ

3.1. Lựa chọn các thông số môi trường cần giám sát:

Lớp học là không gian kín, có mật độ người cao và thời gian sử dụng liên tục trong ngày. Chất lượng không khí trong lớp học có ảnh hưởng trực tiếp đến sức khỏe, khả năng tập trung và hiệu quả học tập của học sinh. Do đó, việc lựa chọn các thông số môi trường phù hợp để giám sát là yếu tố quan trọng trong thiết kế hệ thống mạng cảm biến IoT đo lường và cảnh báo chất lượng không khí học đường.

Để đánh giá toàn diện môi trường không khí, hệ thống cần lựa chọn các thông số đặc trưng, phản ảnh cả điều kiện khi hậu và mức độ ô nhiễm. Trong nghiên cứu này, nhóm tập trung giám sát các thông số đặc trưng cho môi trường lớp học như sau:

- **Nhiệt độ không khí**: Nhiệt độ ảnh hưởng trực tiếp đến cảm giác thoải mái của học sinh và giáo viên trong quá trình học tập. Nhiệt độ quá cao hoặc quá thấp có thể gây mệt mỏi, giảm khả năng tập trung và ảnh hưởng đến hiệu suất học tập. Bên cạnh đó, nhiệt độ cũng tác động đến độ chính xác và độ ổn định của các cảm biến môi trường khác, do đó cần được theo dõi liên tục trong lớp học.
- **Độ ẩm không khí:** Độ ẩm là yếu tố quan trọng phản ánh trạng thái môi trường trong không gian kín. Độ ẩm quá cao có thể gây cảm giác oi bức, tạo điều kiện cho vi khuẩn và nấm mốc phát triển, trong khi độ ẩm quá thấp có thể gây khô da và khó chịu cho đường hô hấp. Việc giám sát độ ẩm trong lớp học giúp đánh giá mức độ thoải mái và điều kiện vệ sinh không khí.
- **Chất lượng không khí tổng hợp (MQ135):** Trong môi trường lớp học kín, nhiều loại khí như CO_2, NH_3, NOx và các khí ô nhiễm khác có thể tích tụ do hoạt động sinh hoạt và thông gió kém. Cảm biến MQ135 được sử dụng để đánh giá tổng hợp mức độ ô nhiễm không khí thông qua sự thay đổi nồng độ các khí này. Mặc dù không đo riêng biệt từng loại khí, thông số này phản ánh xu hướng biến đổi chất lượng không khí và hỗ trợ phát hiện tình trạng môi trường kém thông thoáng.
- **Hợp chất hữu cơ bay hơi (VOC):** VOC là các hợp chất khí phát sinh từ vật liệu xây dựng, bàn ghế, thiết bị điện tử và các sản phẩm sử dụng trong lớp học. Mặc dù thường tồn tại ở nồng độ thấp, VOC có thể tích tụ trong không gian kín và ảnh hưởng đến chất lượng không khí tổng thể. Việc giám sát VOC giúp bổ sung thông tin đánh giá môi trường lớp học và hỗ trợ cảnh báo sớm khi chất lượng không khí suy giảm.

3.1. Lựa chọn và nguyên lý hoạt động của cảm biến:

Trong hệ thống IoT giám sát và cảnh báo chất lượng không khí học đường, việc lựa chọn các phương pháp đo phù hợp đóng vai trò quan trọng nhằm đảm bảo tính khả thi, độ tin cậy và khả năng ứng dụng thực tiễn trong không gian lớp học. Sau khi tham khảo các tài liệu liên quan và điều kiện triển khai thực tế, các thông số môi trường chính được nhóm lựa chọn bao gồm nhiệt độ, độ ẩm và các chỉ số phản ánh chất lượng không khí tổng hợp như khí VOC và các khí ô nhiễm.

Tương ứng với các thông số này, các cảm biến DHT11, MQ135 và SGP30 được sử dụng dựa trên những nguyên lý đo khác nhau, phù hợp với mục tiêu xây dựng hệ thống giám sát chi phí thấp và dễ triển khai trong môi trường học đường.

3.1.1. Cảm biến khí MQ135:



Hình 1: MQ135

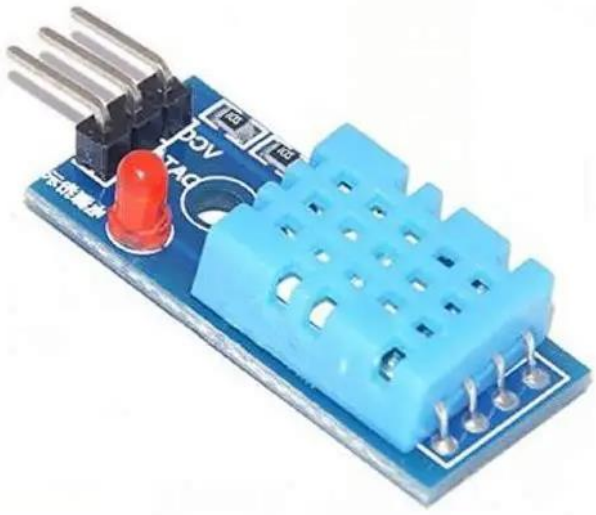
- **Phương pháp đo:** Cảm biến bán dẫn oxit kim loại (MOS – Metal Oxide Semiconductor): Là phương pháp đo nồng độ khí dựa trên sự thay đổi điện trở của vật liệu bán dẫn khi tiếp xúc với các khí trong môi trường.
- **Nguyên lý đo:**
  + MQ135 sử dụng lớp vật liệu SnO_2 (thiếc oxit) được gia nhiệt ở nhiệt độ cao bằng điện trở nung bên trong cảm biến
  + Khi các khí như NH_3, NOx, CO_2, benzen hoặc VOC tiếp xúc với bề mặt cảm biến, chúng gây ra phản ứng oxy hóa – khử
  + Quá trình này làm thay đổi điện trở của lớp bán dẫn

+ Sự thay đổi điện trở được chuyển đổi thành tín hiệu điện áp analog
+ Vi điều khiển đọc tín hiệu này thông qua bộ ADC và xử lý để ước lượng mức độ ô nhiễm không khí

– **Thông số đo được:**
+ Nồng độ khí tổng hợp (NH_3, NOx, CO_2, benzen, khói,…)
+ Giá trị điện áp analog tương ứng với mức độ ô nhiễm
+ Có thể quy đổi tương đối sang CO_2 (ppm) sau khi hiệu chuẩn

– **Đặc điểm phương pháp:**
+ Đo gián tiếp chất lượng không khí, không đặc trưng riêng cho từng loại khí
+ Độ nhạy cao với nhiều loại khí ô nhiễm trong môi trường
+ Giá thành thấp, dễ triển khai trong các hệ thống IoT
+ Độ chính xác phụ thuộc vào quá trình hiệu chuẩn và điều kiện môi trường (nhiệt độ, độ ẩm)
+ Phù hợp cho các ứng dụng giám sát môi trường mang tính tương đối hoặc hệ thống demo

3.1.2. Cảm biến nhiệt độ và độ ẩm DHT11:



Hình 2: DHT11

– **Phương pháp đo:** Cảm biến điện dung và điện trở: sử dụng cảm biến độ ẩm dạng điện dung và cảm biến nhiệt độ dạng điện trở (NTC) để xác định các thông số môi trường.

- **Nguyên lý đo:** DHT11 tích hợp hai thành phần cảm biến:
  + Cảm biến độ ẩm dạng điện dung
  + Cảm biến nhiệt độ dạng điện trở nhiệt (thermistor)
  + Độ ẩm được xác định thông qua sự thay đổi điện dung của lớp vật liệu nhạy ẩm khi hấp thụ hơi nước trong không khí
  + Nhiệt độ được đo dựa trên sự thay đổi điện trở của thermistor theo nhiệt độ môi trường
  + Tín hiệu từ các cảm biến được xử lý bởi bộ vi điều khiển bên trong và xuất ra dưới dạng tín hiệu số (digital)
- **Thông số đo được:**
  + Nhiệt độ (°C)
  + Độ ẩm tương đối (%RH)
- **Đặc điểm phương pháp:**
  + Độ chính xác ở mức trung bình, phù hợp cho các ứng dụng cơ bản
  + Chi phí thấp, dễ sử dụng
  + Tích hợp sẵn xử lý tín hiệu và xuất dữ liệu dạng số
  + Tốc độ cập nhật chậm (~1 Hz)
  + Phù hợp cho hệ thống IoT giám sát môi trường ở mức đơn giản

3.1.3. Cảm biến VOC SGP30:



Hình 3: SGP30

- **Phương pháp đo:** Cảm biến bán dẫn oxit kim loại (MOS – Metal Oxide Semiconductor): Phương pháp Cảm biến bán dẫn oxit kim loại (MOS) là kỹ thuật đo nồng độ khí dựa trên sự thay đổi điện trở của vật liệu bán dẫn khi tiếp xúc với các phân tử khí.
- **Nguyên lý đo:**
  + SGP30 sử dụng lớp vật liệu oxit kim loại được gia nhiệt ở nhiệt độ cao
  + Khi các khí khử (VOC) xuất hiện, chúng phản ứng với bề mặt vật liệu, làm thay đổi điện trở của lớp bán dẫn
  + Sự thay đổi điện trở được chuyển đổi thành tín hiệu số.
  + Bộ xử lý bên trong sử dụng thuật toán nội suy để tính:
    - TVOC (ppb)
    - eCO_2 (ppm – CO_2 tương đương)
- **Thông số đo được:**
  + TVOC (Total Volatile Organic Compounds)
  + eCO_2 (CO_2 tương đương)
- **Đặc điểm phương pháp:**
  + Đo gián tiếp chất lượng không khí
  + Nhạy với các khí phát sinh từ:
  + Hơi người
  + Sơn, keo, bút dạ, chất tẩy rửa
  + Phù hợp đánh giá mức độ ô nhiễm trong lớp học

3.2. Lựa chọn vi điều khiển:

3.2.1. Tổng quan các nền tảng:

- Một số nền tảng vi điều khiển phổ biến được xem xét bao gồm:
  + Arduino Uno: dễ sử dụng nhưng hiệu năng thấp, không có WiFi tích hợp
  + ESP8266: có WiFi nhưng tài nguyên hạn chế
  + Raspberry Pi: hiệu năng cao nhưng chi phí lớn và tiêu thụ điện năng cao

3.2.2. Lý do chọn ESP32:

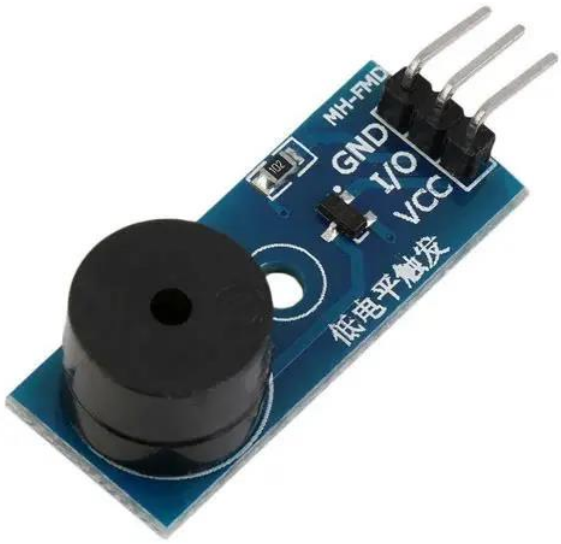
ESP32 được lựa chọn làm bộ điều khiển trung tâm do đáp ứng tốt các yêu cầu của hệ thống IoT:

- Tích hợp sẵn WiFi và Bluetooth
- Vi xử lý hai nhân, tốc độ lên đến 240 MHz
- Khả năng xử lý đồng thời nhiều cảm biến
- Tiêu thụ điện năng thấp, chi phí hợp lý
- ESP32 cho phép hệ thống thu thập, xử lý và truyền dữ liệu theo thời gian thực một cách ổn định

3.2.3. Đa dạng hóa giao tiếp và khả năng mở rộng hệ thống:

Vi điều khiển ESP32 được lựa chọn làm khối xử lý trung tâm nhờ khả năng cung cấp số lượng chân I/O lớn, đồng thời hỗ trợ linh hoạt cả tín hiệu vật lý lẫn các chuẩn giao thức truyền tin đa dạng. Cụ thể trong mô hình nghiên cứu:

- Giao diện tín hiệu tương tự (ADC): Tích hợp bộ chuyển đổi Analog-to-Digital độ phân giải cao để đọc các dải giá trị điện áp biến thiên liên tục từ cảm biến khí Gas (MQ135)
- Giao diện tín hiệu số (Digital I/O - GPIO): Đầu vào (Input) sử dụng chuẩn giao tiếp đơn dây (Single-bus) để thu thập chuỗi dữ liệu số từ cảm biến Nhiệt độ - Độ ẩm (DHT11)
- Đầu ra (Output): Điều khiển mức logic (High/Low) cho cơ cấu chấp hành và thiết bị cảnh báo (Còi Buzzer, Đèn LED)



Hình 4: Buzzer MM-FMD

- Chuẩn giao tiếp I2C: Tạo bus giao tiếp hai dây (SDA, SCL) tin cậy để đọc các thanh ghi dữ liệu môi trường phức tạp từ cảm biến chất lượng không khí (SGP30).
- Dự phòng mở rộng (SPI / UART): ESP32 được tích hợp sẵn các chuẩn giao tiếp tốc độ cao như SPI hay UART. Mặc dù chưa sử dụng hết trong phạm vi đồ án hiện tại, đặc tính này tạo tiền đề vững chắc giúp hệ thống dễ dàng nâng cấp trong tương lai (như tích hợp thêm module màn hình hiển thị LCD/OLED, đầu đọc thẻ nhớ hoặc module kết nối LoRa) mà không cần thay đổi kiến trúc phần cứng tổng thể.

PHẦN 4. XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG DEMO

4.1. Thiết kế phần cứng hệ thống:

Hệ thống phần cứng được thiết kế theo mô hình mạng cảm biến IoT, bao gồm vi điều khiển trung tâm, các cảm biến môi trường và các thiết bị cảnh báo.

Trong đó, vi điều khiển ESP32 đóng vai trò trung tâm, thực hiện thu thập và xử lý dữ liệu từ các cảm biến: DHT11 (nhiệt độ, độ ẩm), MQ135 (chất lượng không khí tổng hợp) và SGP30 (VOC).

Các cảm biến được kết nối với ESP32 thông qua các giao thức khác nhau:

- DHT11 sử dụng giao tiếp digital
- MQ135 xuất tín hiệu analog và được đọc qua chân ADC
- SGP30 sử dụng giao tiếp I2C

Ngoài ra, hệ thống còn tích hợp các thiết bị cảnh báo như LED và buzzer nhằm thông báo khi chất lượng không khí vượt ngưỡng cho phép. Toàn bộ hệ thống được cấp nguồn ổn định thông qua nguồn DC 5V.

Thiết kế phần cứng đảm bảo tính đơn giản, chi phí thấp và phù hợp triển khai trong môi trường lớp học.

4.2. Xây dựng phần mềm nhúng (Firmware):

Phần mềm nhúng được phát triển trên nền tảng ESP32 nhằm thực hiện các chức năng chính: thu thập dữ liệu, xử lý và truyền dữ liệu lên hệ thống lưu trữ.

Quy trình hoạt động của firmware bao gồm:

- Khởi tạo các cảm biến (DHT11, MQ135, SGP30)
- Thiết lập kết nối WiFi
- Đọc dữ liệu định kỳ từ các cảm biến
- Xử lý dữ liệu (lọc nhiễu, chuẩn hóa giá trị)
- So sánh với các ngưỡng cảnh báo
- Gửi dữ liệu lên máy chủ

Chương trình được thiết kế theo vòng lặp liên tục, đảm bảo hệ thống hoạt động theo thời gian thực và ổn định trong quá trình vận hành.

4.3. Truyền và lưu trữ dữ liệu (InfluxDB):

Dữ liệu từ ESP32 được truyền lên máy chủ thông qua giao thức HTTP hoặc MQTT thông qua kết nối WiFi.

Sau khi nhận dữ liệu, hệ thống lưu trữ sử dụng InfluxDB – một cơ sở dữ liệu tối ưu cho dữ liệu chuỗi thời gian. Các thông số như nhiệt độ, độ ẩm, chỉ số khí tổng hợp (MQ135) và TVOC (SGP30) được lưu trữ theo mốc thời gian cụ thể.

Việc sử dụng InfluxDB giúp hệ thống dễ dàng truy xuất dữ liệu lịch sử, phục vụ cho việc phân tích và hiển thị trên dashboard.

4.4. Xây dựng dashboard giám sát (Grafana):

Dữ liệu lưu trữ trong InfluxDB được trực quan hóa thông qua nền tảng Grafana. Dashboard được thiết kế nhằm hiển thị các thông số môi trường theo thời gian thực dưới dạng biểu đồ trực quan.

Các chức năng chính của dashboard bao gồm:

- Hiển thị giá trị tức thời của các thông số
- Biểu đồ biến đổi theo thời gian
- Phân vùng cảnh báo theo màu sắc (xanh – vàng – đỏ)
- Theo dõi xu hướng thay đổi chất lượng không khí

Dashboard giúp người dùng dễ dàng quan sát và đánh giá chất lượng môi trường lớp học một cách trực quan và liên tục.

4.5. Luồng hoạt động của hệ thống:

Hệ thống hoạt động theo quy trình khép kín từ thu thập dữ liệu đến hiển thị và cảnh báo, cụ thể như sau:

- Các cảm biến DHT11, MQ135 và SGP30 đo các thông số môi trường
- ESP32 thu thập và xử lý dữ liệu
- Dữ liệu được truyền qua WiFi lên máy chủ
- InfluxDB lưu trữ dữ liệu theo thời gian
- Grafana hiển thị dữ liệu dưới dạng dashboard
- Khi giá trị vượt ngưỡng, hệ thống kích hoạt cảnh báo (LED, buzzer)

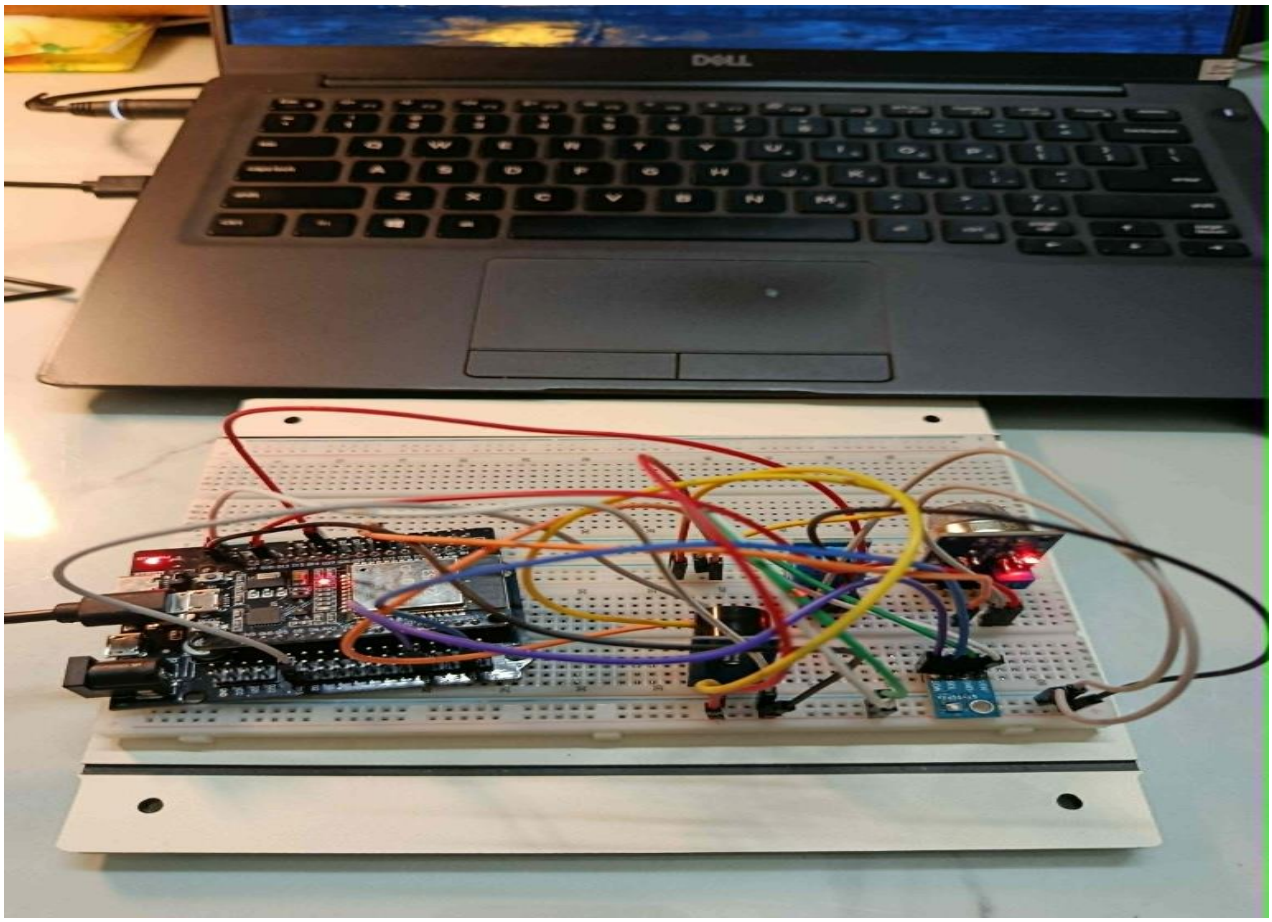
Luồng hoạt động này đảm bảo khả năng giám sát liên tục và phản hồi kịp thời khi chất lượng không khí thay đổi.

PHẦN 5. CÁCH LẮP MẠCH, THIẾT LẬP, HIỂN THỊ CÁC CHỈ SỐ TRÊN HOME ASSISTANT VÀ THÔNG BÁO RA MÀN HÌNH ĐIỆN THOẠI

Để hiện thực hóa mô hình lý thuyết thành một trạm quan trắc vi khí hậu hoạt động thực tiễn, quá trình triển khai được tiến hành tuần tự qua 5 bước cốt lõi: lắp ráp phần cứng, lập trình vi điều khiển, thiết lập máy chủ ảo, cấu hình giao diện và thiết lập tự động hóa cảnh báo.

5.1. Lắp ráp và đấu nối sơ đồ phần cứng:

Hệ thống sử dụng vi điều khiển ESP32 làm khối xử lý trung tâm. Trong giai đoạn thử nghiệm (Prototyping), các linh kiện được liên kết trên Breadboard (Testboard) nhằm tối ưu hóa việc chia sẻ nguồn cấp chung (3.3V và 5V) cũng như dễ dàng tinh chỉnh sơ đồ chân (Pinout).



Hình 5: Nguyên mẫu phần cứng Trạm đo không khí IoT được lắp ráp trên Breadboard

- **Quy chuẩn đấu nối linh kiện:**
  - **Cảm biến SGP30 (eCO2, TVOC):** Giao tiếp qua chuẩn I2C. Chân SCL nối với GPIO 22, chân SDA nối với GPIO 21 của ESP32. Cấp nguồn 3.3V.

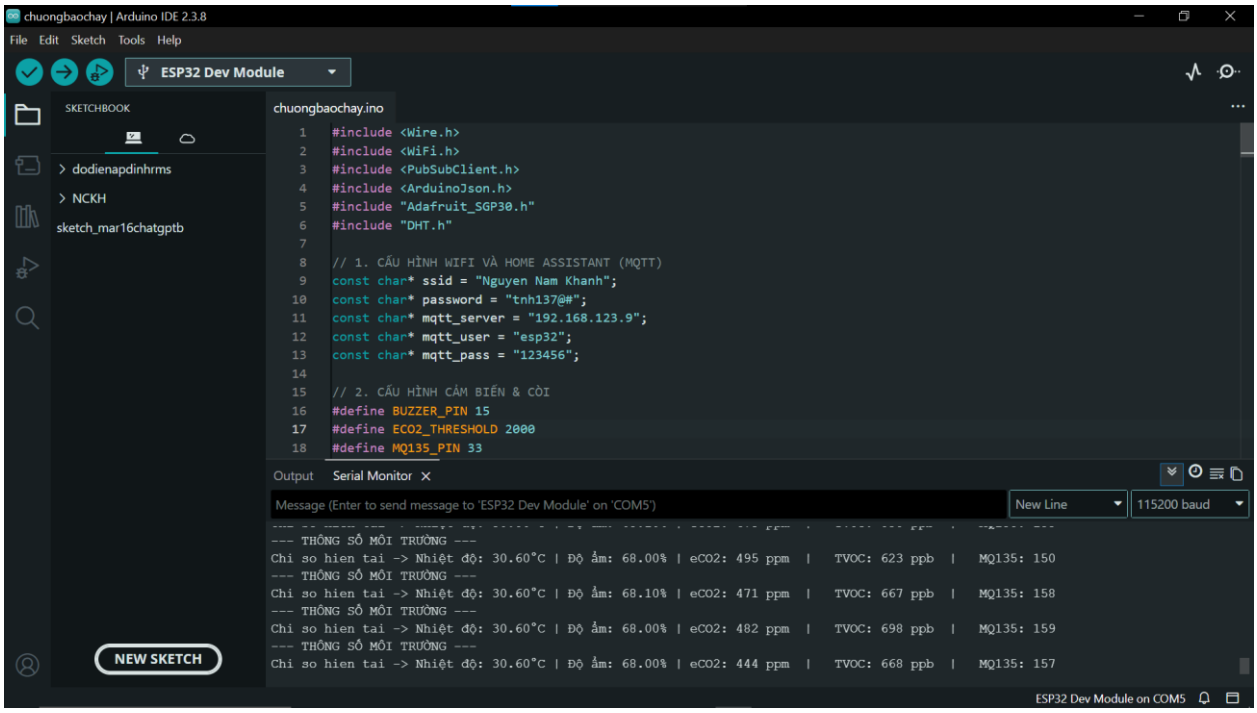
+ **Cảm biến MQ135 (Khí Gas):** Đọc tín hiệu Analog. Chân đầu ra A0 kết nối với GPIO 33. Cấp nguồn 5V.

+ **Cảm biến DHT11 (Nhiệt độ, Độ ẩm):** Giao tiếp Digital (Single-bus). Chân Data kết nối với GPIO 14.

+ **Module Cảnh báo (Buzzer/LED):** Kết nối với GPIO 15 để nhận tín hiệu kích hoạt mức cao (High) khi có rủi ro.

5.2. Lập trình và nạp mã nguồn (Firmware) cho ESP32

Sau khi hoàn thiện phần cứng, hệ thống được lập trình bằng ngôn ngữ C/C++ thông qua môi trường phát triển tích hợp Arduino IDE.

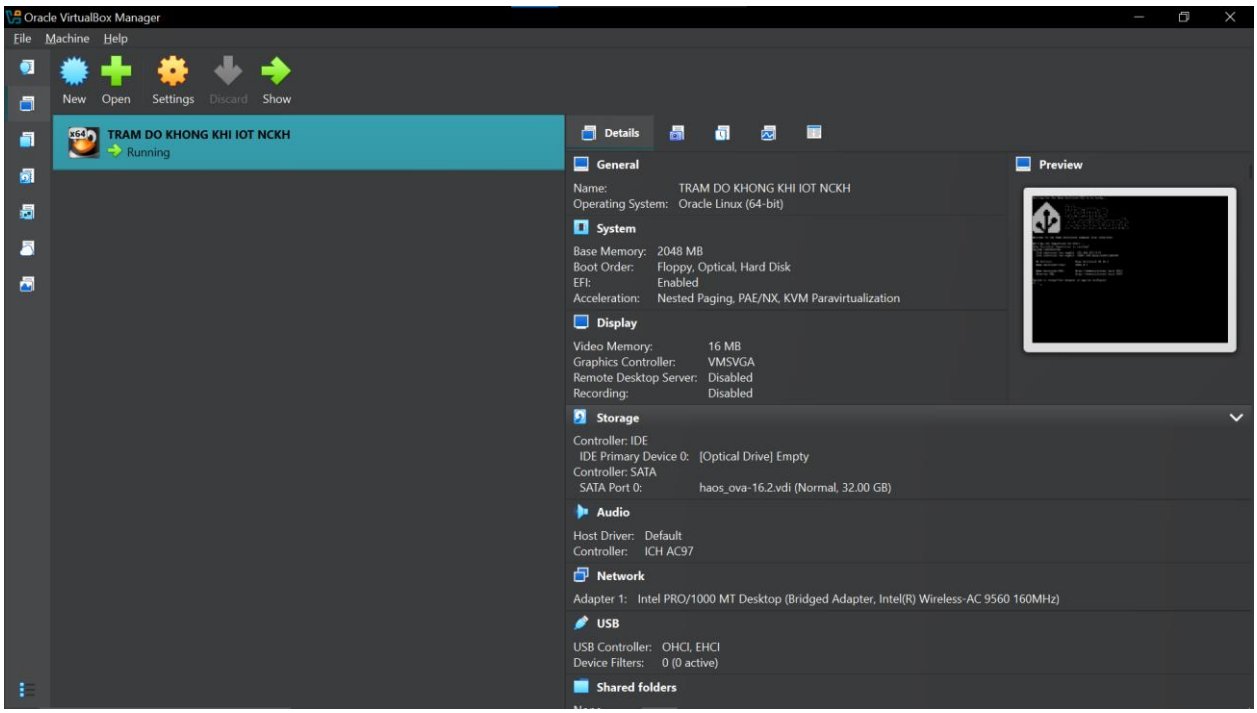


Hình 6: Giao diện nạp mã nguồn và kiểm thử dữ liệu thô trên Serial Monitor

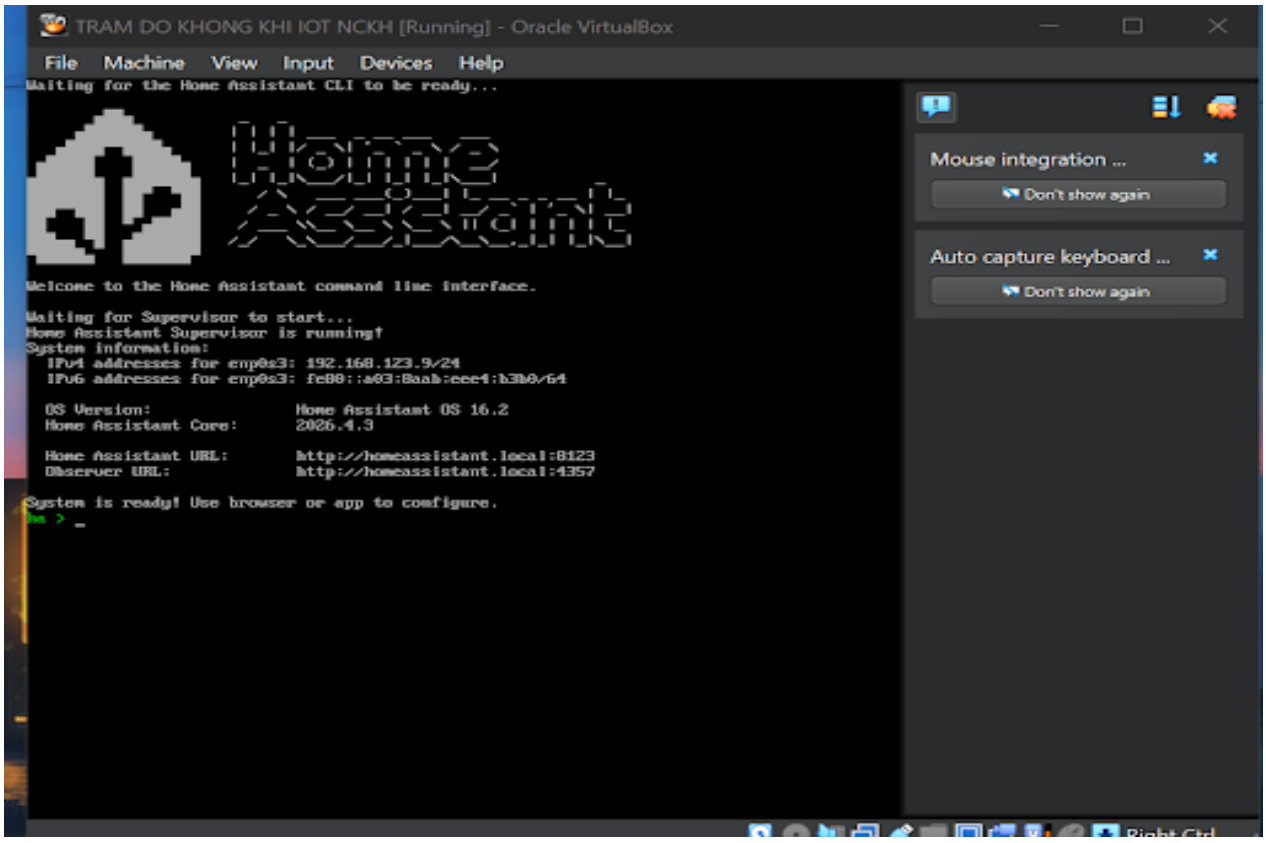
Mã nguồn được cấu trúc thành các khối chức năng rõ ràng: Khởi tạo WiFi, kết nối MQTT Broker, đọc dữ liệu định kỳ từ các cảm biến và đóng gói thành bản tin JSON. Như quan sát trên màn hình Serial Monitor (Hình 5.2), ESP32 đã đọc và giải mã thành công toàn bộ 5 dải thông số môi trường với tốc độ baud 115200, tạo cơ sở dữ liệu thô (raw data) sẵn sàng truyền tải lên đám mây.

5.3. Cài đặt và cấu hình máy chủ cục bộ Home Assistant:

Để đảm bảo tính bảo mật và tốc độ phản hồi thời gian thực (không phụ thuộc vào server quốc tế), nhóm nghiên cứu quyết định tự lưu trữ máy chủ Home Assistant (Self-hosted) thông qua phần mềm ảo hóa Oracle VirtualBox.



Hình 7: Cấu hình máy chủ ảo VirtualBox



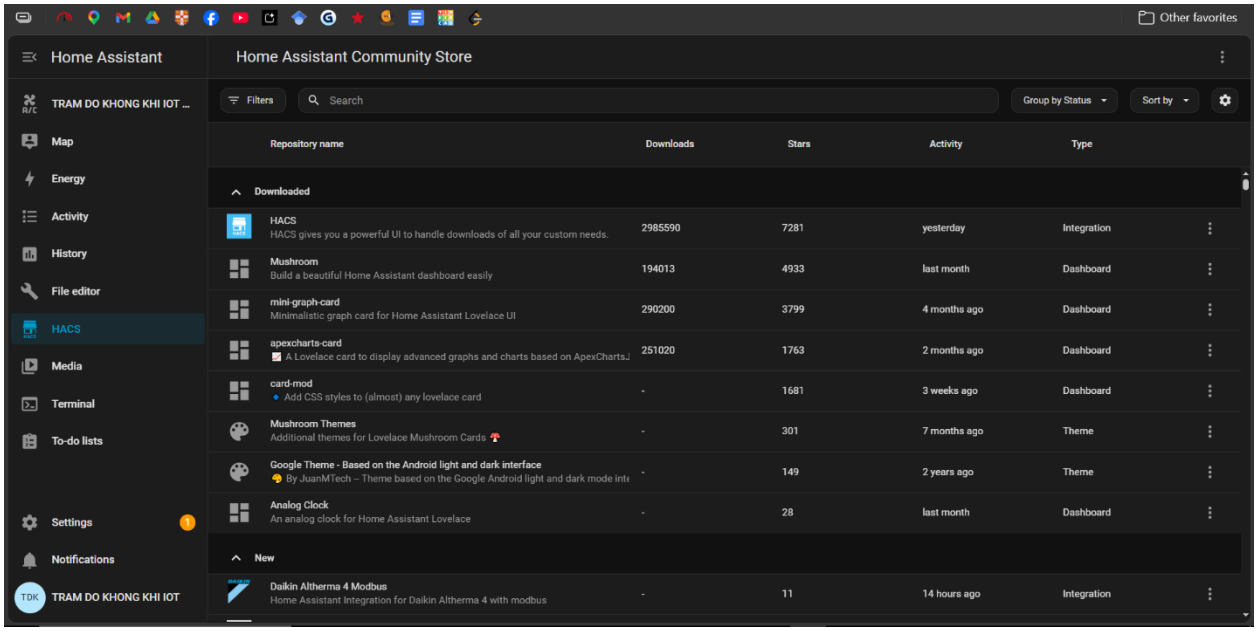
Hình 8: Giao diện khởi động Home Assistant CLI

– **Tạo môi trường ảo:** Hệ điều hành Home Assistant OS được nạp vào VirtualBox với cấu hình cấp phát 2GB RAM và kết nối mạng dạng Bridged Adapter để nằm cùng dải mạng nội bộ (LAN) với vi điều khiển ESP32.

– **Vận hành hệ thống:** Khi khởi động, Home Assistant cung cấp một địa chỉ IPv4 cục bộ (ví dụ: 192.168.x.x:8123 như Hình 5.4). Máy chủ này đóng vai trò là "Bộ não trung tâm", cài đặt sẵn Add-on Mosquitto broker để lắng nghe các gói tin MQTT do ESP32 gửi lên.

5.4. Tùy biến và thiết kế giao diện hiển thị (Dashboard UI) thông qua HACS:

Giao diện mặc định của Home Assistant thường khá đơn điệu và giới hạn về khả năng tùy biến. Để xây dựng một bảng điều khiển (Dashboard) trực quan, hiện đại và đáp ứng khả năng cảnh báo bằng màu sắc, nhóm nghiên cứu đã tích hợp cửa hàng ứng dụng cộng đồng HACS (Home Assistant Community Store).



Hình 9: Quản lý các gói giao diện mở rộng thông qua thư viện HACS

Thông qua HACS, hệ thống được cài đặt các thư viện Front-end chuyên sâu:

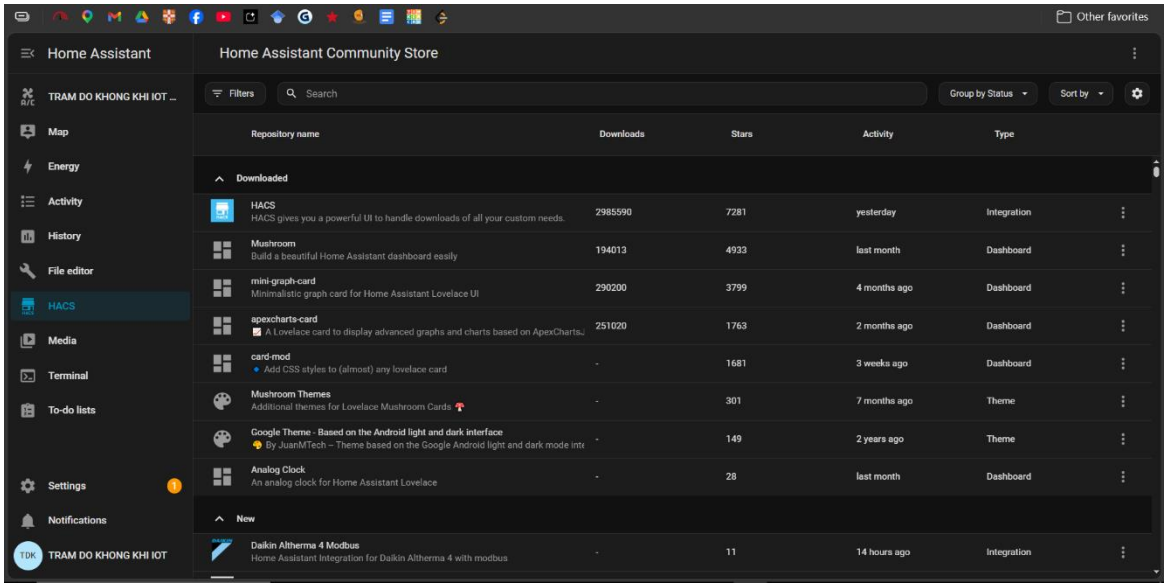
– **Mushroom & Mushroom Themes:** Cung cấp bộ thẻ (Cards) giao diện hiện đại, tối giản theo phong cách thẻ bo góc, thân thiện với thao tác cảm ứng trên thiết bị di động.

– **Card-mod:** Thư viện can thiệp mã CSS tùy chỉnh, cho phép nhúng trực tiếp ngôn ngữ lập trình Jinja2 vào giao diện. Nhờ Card-mod, thẻ hiển thị có khả năng tự động đọc điểm số (Score) của 5 cảm biến, từ đó thay đổi màu nền (Gradient background), viền phát sáng (Box-shadow) và biểu tượng (Icon) theo thời gian thực mà không cần tải lại trang.

– **Apexcharts-card & Mini-graph-card**: Hỗ trợ render các biểu đồ lịch sử chất lượng cao, giúp vẽ lại đường đi của nồng độ khí Gas, eCO2 và TVOC một cách sắc nét và chi tiết.

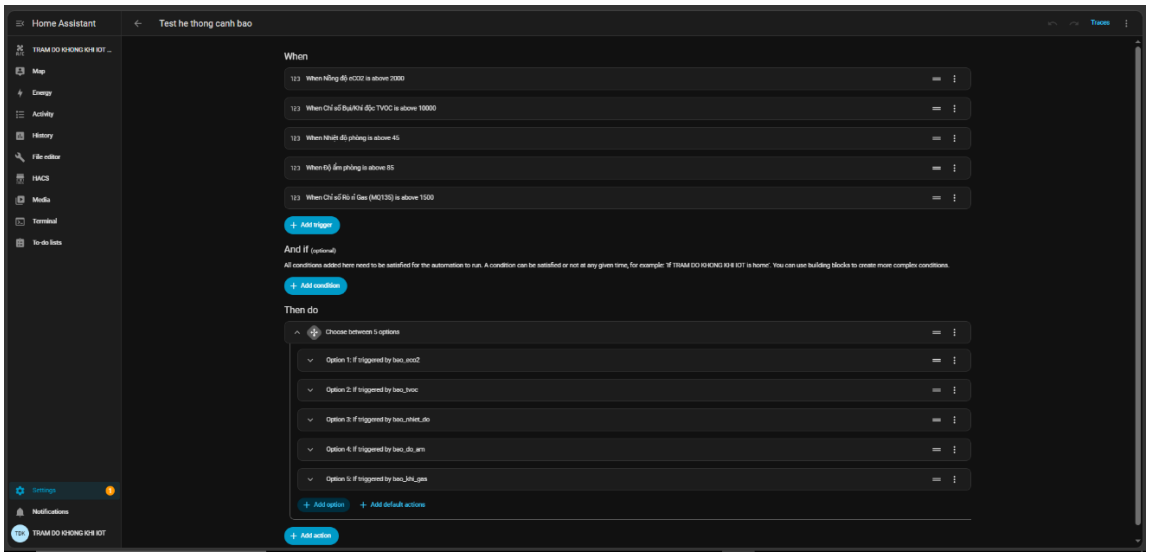
5.5. Thiết lập tự động hóa (Automations) và hệ thống cảnh báo đa cấp độ

Để giải quyết bài toán giám sát từ xa, hệ thống được thiết lập các kịch bản Tự động hóa (Automations) chạy ngầm 24/7. Cấu trúc logic được chia thành hai kịch bản đối lập nhưng có tính liên kết chặt chẽ: Cảnh báo rủi ro và Xác nhận an toàn.



Hình 10: Quản lý các kịch bản tự động hóa trên Home Assistant

– **Kịch bản phát hiện rủi ro (Test hệ thống cảnh báo):** Kịch bản này đóng vai trò như một lính gác, phản ứng ngay lập tức khi phát hiện bất thường từ môi trường.

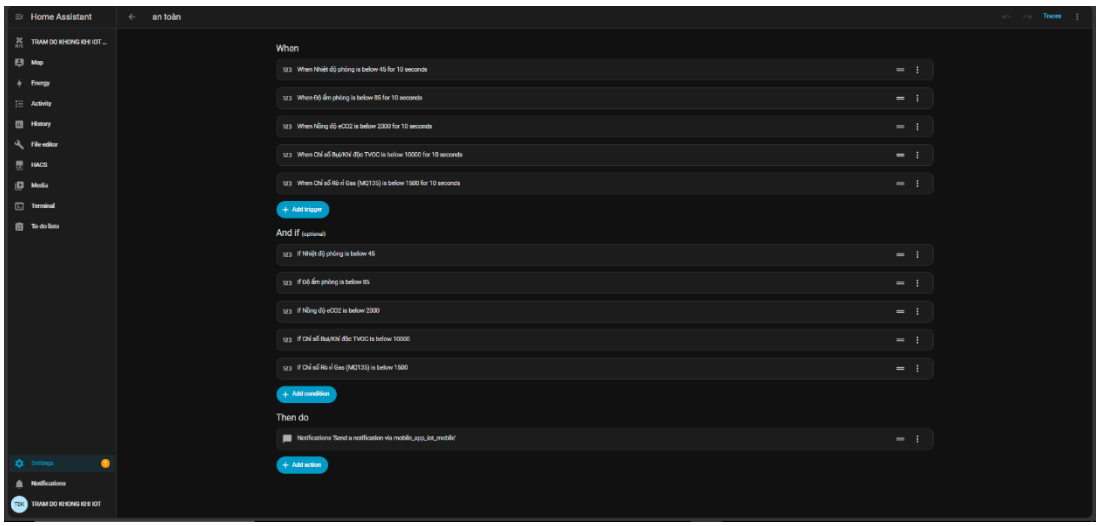


Hình 11: Cấu trúc Trigger đa luồng xử lý cảnh báo nguy hiểm

+ **Khối kích hoạt (When / Triggers):** Thiết lập 5 điều kiện kích hoạt song song tương ứng với ngưỡng nguy hiểm của 5 cảm biến (VD: Nồng độ eCO2 > 2000 ppm; Khí Gas > 1500; TVOC > 10000...). Bất kỳ chỉ số nào vượt ngưỡng đều lập tức kích hoạt luồng tự động hóa.

+ **Khối hành động (Then do / Actions):** Sử dụng thuật toán rẽ nhánh Choose between 5 options. Hệ thống sẽ tự động đối chiếu xem cảm biến nào vừa gây ra kích hoạt (Trigger ID), từ đó cấu trúc nội dung tin nhắn tương ứng (Ví dụ: "Cảnh báo rò rỉ khí gas" hoặc "Cảnh báo nhiệt độ cao") để gửi cảnh báo đẩy (Push Notification) đến điện thoại người quản lý.

– **Kịch bản phục hồi và Xác nhận an toàn :** Trong thực tế, tín hiệu cảm biến ở ranh giới ngưỡng thường xuyên dao động (Flapping), dẫn đến tình trạng hệ thống gửi tin nhắn "An toàn - Nguy hiểm" liên tục gây nhiễu (Spam) cho người dùng. Kịch bản "An toàn" được thiết kế đặc biệt để khắc phục triệt để vấn đề này.



Hình 12: Cấu trúc logic độ trễ và kiểm tra chéo trong kịch bản xác nhận an toàn

+ **Cơ chế chống nhiễu (Hysteresis & Time Delay):** Ở phần Trigger (When), thông số không chỉ cần hạ xuống dưới ngưỡng an toàn (VD: Gas < 1500), mà còn bắt buộc phải duy trì trạng thái đó liên tục trong khoảng thời gian 10 giây (for 10 seconds). Nếu trong 10 giây này, tín hiệu vọt lên lại, bộ đếm sẽ bị hủy.

+ **Cơ chế kiểm tra chéo toàn hệ thống (And if / Conditions):** Đây là chốt chặn logic quan trọng nhất. Khi một chỉ số (VD: khí Gas) đã về ngưỡng an toàn, hệ thống không thông báo ngay. Nó phải quét qua khối điều kiện And if để kiểm tra 4 thông số còn lại. Chỉ khi **tất cả** các yếu tố (Nhiệt, Ẩm, eCO2, TVOC) đều đang nằm ở mức an toàn, kịch bản mới được phép đi tiếp.

+ **Hành động (Action):** Gửi thông báo xanh với nội dung: "Trạng thái: AN TOÀN - HỆ THỐNG AN TOÀN: Mọi chỉ số đã ổn định".

Nhờ thiết lập logic đa tầng này, trạm quan trắc hoạt động cực kỳ mượt mà, cung cấp thông tin chính xác, đúng thời điểm và hạn chế tối đa các cảnh báo giả.

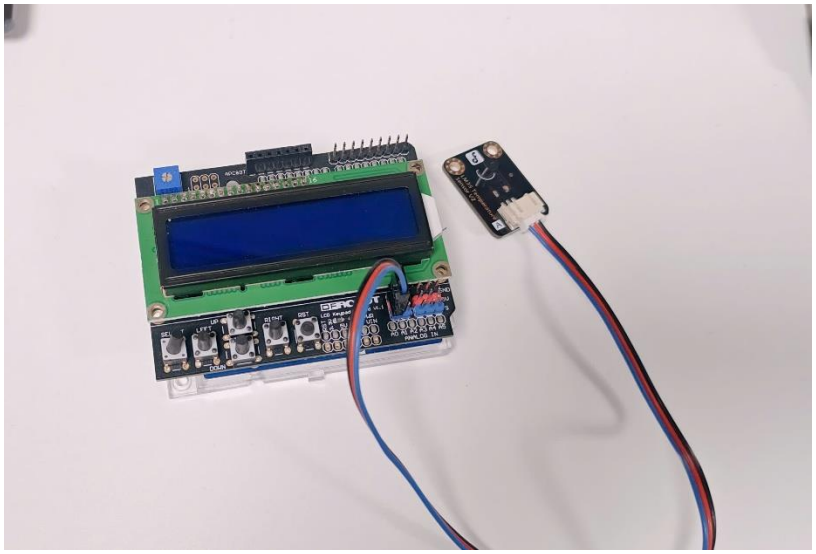
PHẦN 6. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

6.1. Đo nhiệt độ và độ ẩm:

6.1.1. Kết quả nhiệt độ (Giai đoạn 1) :

Để tích kiệm kinh phí, nhóm đã xây dựng code phần mềm trên mạch Arduino Uno Wifi Rev 2 để tạo được 1 phần mềm giám sát môi trường demo với chức năng giám sát nhiệt độ.

Nhóm đã mô phỏng Proteus và thực hành mô phỏng lại trên thực tế với vi xử lý là Arduino Uno Wifi Rev 2 và cảm biến demo là LM35 chuyên dùng để đo nhiệt độ.



Hình 13: Hình ảnh mạch demo đo nhiệt độ

Đầu tiên, sau khi được cấp điện, mạch Arduino (đã được input code) sẽ đo nhiệt độ môi trường xung quanh thông qua cảm biến LM35 được cắm ở cổng A1, chuyển đổi sự thay đổi nhiệt độ thành điện áp analog tuyến tính với hệ số 10 mV/°C và được ghi lại và gửi qua cổng Serial; đồng thời, nhiệt độ trực tiếp cũng được hiển thị trên màn hình led được gắn trên mạch. Nếu nhiệt độ quá nóng hoặc quá lạnh (dưới 20 độ và trên 28 độ), màn hình led sẽ hiển thị cảnh báo (Hình 14 và hình 15)



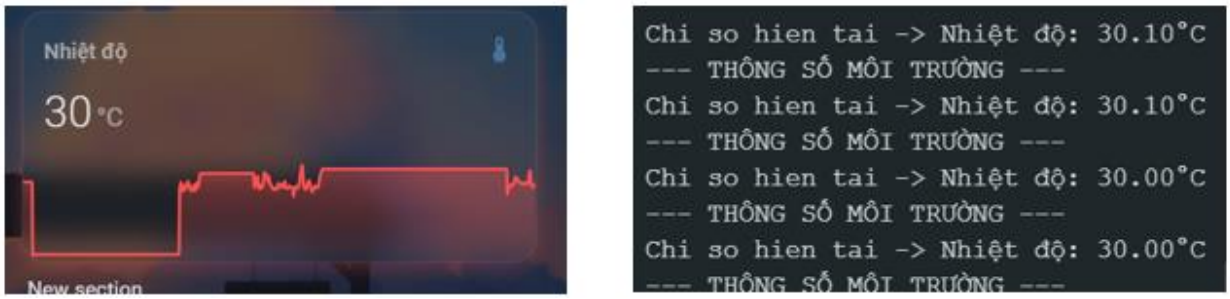
Hình 14: Mạch hiển thị cảnh báo khi môi trường nóng



Hình 15: Mạch hiển thị nhiệt độ khi môi trường bình thường

Nhiệt độ sau khi được ghi lại, sẽ được Arduino gửi lên cơ sở đám mây (InfluxDB) và được mô phỏng trực quan qua biểu đồ

6.1.2. Hoàn thiện và Tích hợp hệ thống tổng thể (Giai đoạn 2):



Hình 16: Dữ liệu Nhiệt độ qua Home Assistant và Arduino IDE

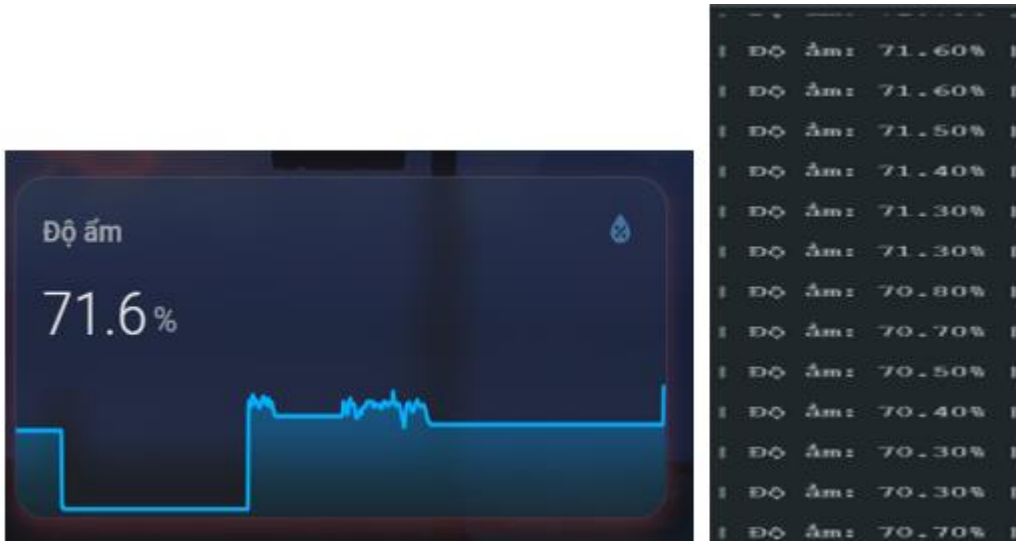
Sau khi thử nghiệm thành công cơ chế cảnh báo và truyền lặp IoT ở Giai đoạn 1, nhóm nhận thấy mô hình cơ sở vẫn tồn tại một số hạn chế để áp dụng vào thực tiễn đo lường không khí toàn diện: cảm biến LM35 chỉ đo được nhiệt độ, tín hiệu Analog dễ bị nhiễu khi kéo dây dài, và nền tảng InfluxDB khó kết hợp các kịch bản tự động hóa nhà

thông minh. Do đó, ở mô hình hoàn thiện cuối cùng, nhóm đã tiến hành nâng cấp phần cứng:

- **Chuyển đổi cảm biến:** Thay thế LM35 bằng cảm biến DHT11. Sự thay đổi này giúp hệ thống thu thập đồng thời cả Nhiệt độ và Độ ẩm (thông số bắt buộc để phân tích cùng chỉ số bụi mịn và khí độc), đồng thời chuyển sang dùng tín hiệu Digital (Single-bus) giúp chống nhiễu tốt hơn.
- **Nâng cấp Vi điều khiển và Nền tảng:** Chuyển đổi sang khối xử lý trung tâm ESP32 với khả năng đa nhiệm mạnh mẽ hơn, kết nối trực tiếp với máy chủ Home Assistant qua MQTT. Hệ thống mới kế thừa toàn bộ tư duy cảnh báo ở Giai đoạn 1, nhưng mở rộng quy mô hiển thị lên giao diện Dashboard chuyên nghiệp thay vì chỉ dùng màn hình LCD đơn giản.

6.2. Kết quả đo độ ẩm (Sử dụng cảm biến DHT11):

- **Mục đích thực nghiệm:** Kiểm chứng độ tin cậy của cảm biến DHT11 trong việc đo lường độ ẩm tương đối (%RH) của môi trường thực tế. Đồng thời, đánh giá độ trễ (latency) và tính toàn vẹn của dữ liệu trong quá trình đồng bộ hóa từ phần cứng (thu thập qua chuẩn Single-bus, xử lý tại vi điều khiển) lên nền tảng IoT (truyền qua MQTT tới Home Assistant), thông qua phương pháp đối chiếu trực tiếp giữa Serial Monitor và giao diện Web.



Hình 17: Dữ liệu đo độ ẩm qua Home Assistant và Arduino IDE

- **Phân tích kết quả đo lường:** Dựa trên kết quả thu nhận từ nguyên mẫu thực nghiệm, phân hệ đo lường độ ẩm môi trường đã ghi nhận những thông số tích cực về tính ổn định và khả năng đồng bộ:
  + **Độ chính xác và đồng bộ dữ liệu thời gian thực (Real-time Synchronization):** Tại cùng một thời điểm ghi nhận, dữ liệu thô (raw data)

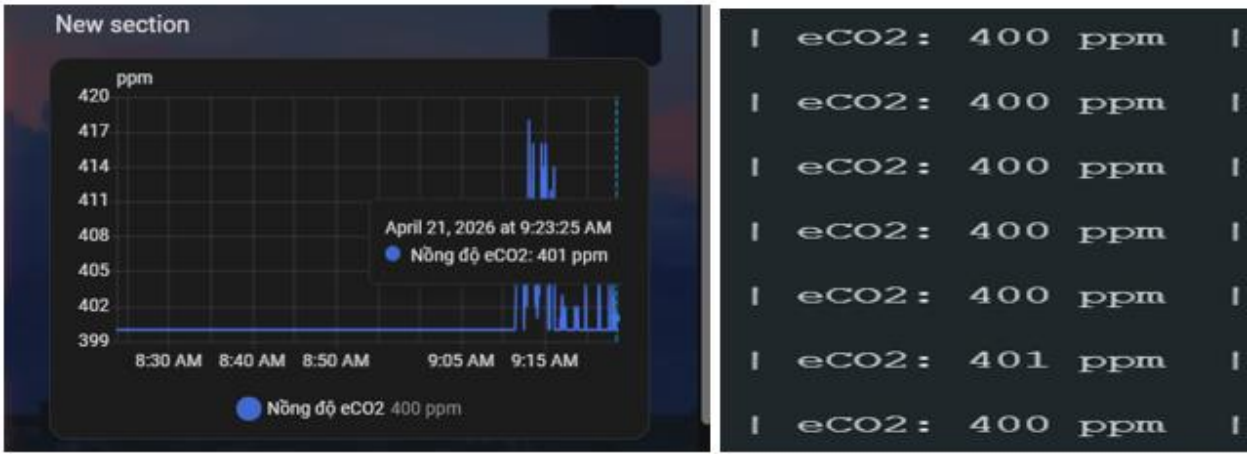
xử lý tại vi điều khiển ESP32 xuất ra cổng Serial đạt mức 71.60%. Cùng lúc đó, giá trị hiển thị trên giao diện điều khiển trung tâm Home Assistant cũng cập nhật ≈ 71.6%. Điều này minh chứng cho quá trình đóng gói và truyền tải bản tin JSON qua giao thức MQTT hoạt động vô cùng trơn tru, không xảy ra hiện tượng suy hao dữ liệu hay độ trễ tín hiệu (low latency) trong hệ thống mạng cục bộ.

+ **Trực quan hóa sự biến thiên môi trường:** Không dừng lại ở việc hiển thị thông số tức thời, biểu đồ dạng đường (Sparkline) tích hợp trên thẻ giao diện Home Assistant đã biểu diễn rõ ràng lịch sử biến thiên của độ ẩm phòng. Biểu đồ trong hình cho thấy khả năng bám sát sự thay đổi của môi trường: từ giai đoạn duy trì trạng thái nền (Baseline), pha sụt giảm đột ngột, cho đến quá trình phục hồi và dao động nhẹ. Nhờ đó, người quản lý có cái nhìn tổng quan về xu hướng vi khí hậu tại khu vực thực nghiệm.

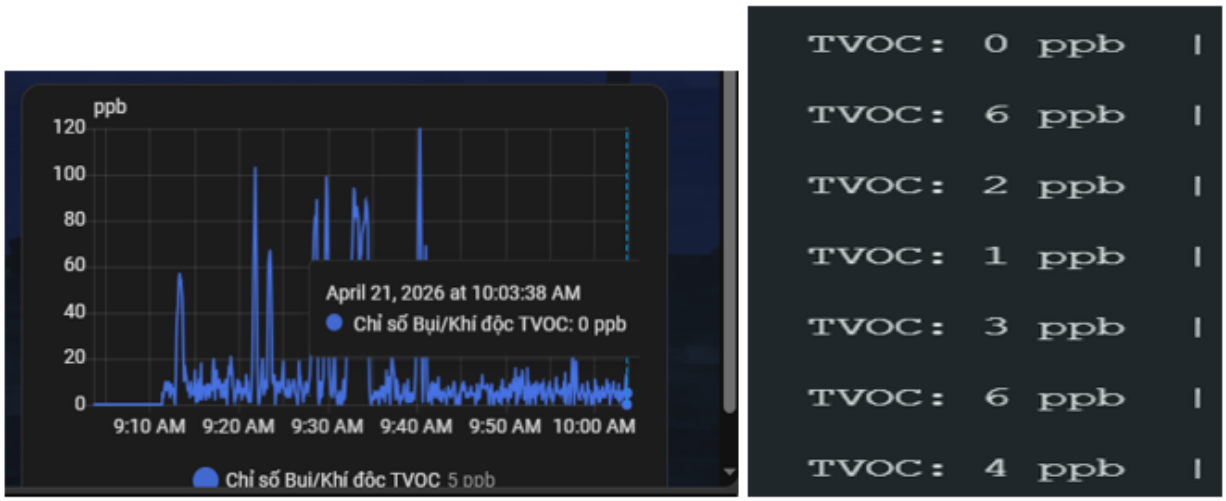
– **Đánh giá chung:** Phân hệ đo lường sử dụng cảm biến DHT11 kết hợp giao thức MQTT đã đáp ứng xuất sắc các tiêu chí kỹ thuật của đề tài. Dữ liệu đầu ra thể hiện tính tin cậy cao, định dạng hiển thị trực quan, tạo cơ sở dữ liệu vững chắc để kích hoạt các kịch bản tự động hóa (Automations) cảnh báo an toàn khi thời tiết diễn biến xấu.

6.3. Kết quả giám sát nồng độ eCO2 và TVOC (Sử dụng cảm biến SGP30):

– **Mục đích thực nghiệm:** Đánh giá độ nhạy, tính ổn định và khả năng đa nhiệm của cảm biến SGP30 thông qua chuẩn giao tiếp I2C trong việc đo lường đồng thời hai dải thông số: Nồng độ CO2 tương đương (eCO2) và Tổng hợp chất hữu cơ bay hơi (TVOC). Đồng thời, kiểm chứng khả năng trực quan hóa và thuật toán phân cấp cảnh báo (Scoring Algorithm) trên máy chủ Home Assistant.



Hình 18: Dữ liệu CO2 qua Home Assistant và Arduino IDE



Hình 19: Dữ liệu TVOC qua Home Assistant và Arduino IDE

– **Phân tích kết quả đo lường:** Sử dụng cảm biến SGP30 giao tiếp qua chuẩn I2C, hệ thống tiến hành thu thập đồng thời hai thông số môi trường quan trọng. Kết quả trích xuất từ mô hình thực nghiệm cho thấy các đặc tính kỹ thuật vượt trội:

+ **Độ ổn định đường nền độc lập (Independent Baseline Stability):** Kênh eCO2: Theo tiêu chuẩn môi trường, không khí trong lành có nồng độ CO2 xấp xỉ 400 ppm. Cả Serial Monitor và biểu đồ Home Assistant đều ghi nhận đường nền phẳng lì và cực kỳ ổn định ở mốc 400 - 401 ppm.

+ **Kênh TVOC:** Serial Monitor ghi nhận giá trị dao động ở mức rất thấp, từ 0 đến 6 ppb. Biểu đồ Home Assistant cũng phản ánh chính xác trạng thái này (hiển thị mốc 0 ppb và 5 ppb tại thời điểm trỏ chuột). Điều này chứng minh thuật toán bù trừ nội tại của cảm biến SGP30 đã phân tách và hiệu chuẩn cực kỳ chuẩn xác cho từng loại khí.

+ **Đồng bộ hóa và xử lý đa luồng (Multithreading Synchronization):** Việc ESP32 phải liên tục yêu cầu và bóc tách 2 gói dữ liệu khác nhau từ cùng một bus I2C không làm chậm hệ thống. Dữ liệu thô từ Serial Monitor và giao diện máy chủ MQTT khớp nhau hoàn toàn theo thời gian thực, không xảy ra hiện tượng "nghẽn cổ chai" (bottleneck) hay rớt gói tin.

+ **Độ nhạy với biến thiên vi mô (Micro-fluctuations):** Trên biểu đồ eCO2 (khoảng 9:15 AM), hệ thống ghi nhận pha tăng đột biến biên độ nhỏ lên mức ~418 ppm, phản ánh độ nhạy của màng đo khi có người (nguồn phát thải CO2 do hô hấp) tiến lại gần trạm đo.

+ Trong khi đó, biểu đồ TVOC cho thấy sự dao động nhạy bén hơn với các chóp nhọn (spikes) vọt lên mức ~60 đến ~120 ppb rải rác theo thời gian. Sự khác biệt hình thái biểu đồ này minh chứng rõ nét cho bản chất vật lý của khí: eCO2 thường tích tụ chậm, trong khi TVOC (hợp chất hữu cơ bay hơi từ mồ hôi, vật liệu, mỹ phẩm) dễ dàng biến thiên liên tục theo các luồng gió nhỏ trong phòng.

– **Nhận xét và đánh giá:**

+ **Về đáp ứng phần cứng:** Cảm biến SGP30 thể hiện hiệu năng xuất sắc khi truyền tải song song hai luồng dữ liệu qua bus I2C (chân SDA/SCL) mà không xảy ra hiện tượng "nghẽn cổ chai". Đặc biệt, thực nghiệm số 4 đã chứng minh màng lọc MOX nội tại của SGP30 có khả năng bóc tách độc lập giữa gốc hóa chất (TVOC) và gốc Hydrogen (quy đổi eCO2).

+ **Về logic phần mềm và giao diện:** Biểu đồ lịch sử trên thẻ hiển thị đã trực quan hóa thành công sự biến thiên độc lập của hai chỉ số. Thuật toán chấm điểm (Scoring Algorithm) bằng Jinja2 trên thẻ Mushroom Card xử lý logic mượt mà: hệ thống tự động quét qua toàn bộ 5 biến môi trường, chỉ cần eCO2 hoặc TVOC chạm ngưỡng báo động, giao diện sẽ lập tức đồng bộ đổi màu cảnh báo mà không xảy ra xung đột logic.

⇒ **Kết luận:** Phân hệ giám sát đáp ứng xuất sắc yêu cầu kiểm soát "hội chứng nhà kín" (Sick Building Syndrome), giúp người quản lý nhận diện chính xác nguồn ô nhiễm đến từ sự bí bách (eCO2) hay từ hóa chất tẩy rửa, vật liệu nội thất (TVOC).

6.4. Kết quả đo MQ135:

– **Mục đích thực nghiệm:** Đánh giá độ nhạy và tốc độ phản hồi của cảm biến MQ135 trong việc phát hiện nồng độ khí dễ cháy (Combustible gases). Đồng thời, kiểm chứng tính toàn vẹn của dữ liệu (data integrity) trong quá trình truyền tải từ phần cứng lên đám mây, thông qua phương pháp đối chiếu chéo giữa cổng Serial Monitor của vi điều khiển và biểu đồ trực quan hóa trên giao diện Home Assistant.



Hình 20: Dữ liệu khí gas qua Home Assistant và Arduino IDE

- **Phân tích kết quả đo lường:** Dựa vào tập dữ liệu thu thập được từ nguyên mẫu, hệ thống giám sát khí Gas thể hiện hai ưu điểm kỹ thuật nổi bật:
  + **Đồng bộ hóa dữ liệu thời gian thực (Real-time Synchronization):** Trong trạng thái Baseline an toàn, giá trị thô ghi nhận tại Serial Monitor dao động trong khoảng 144 - 157. Tại một thời điểm lấy mẫu ngẫu nhiên, Serial Monitor xuất giá trị 149, và ngay lập tức giao diện Home Assistant cũng hiển thị thông số chính xác 149 trên thẻ MQ135 Gas Level. Sự trùng khớp tuyệt đối này khẳng định quá trình đóng gói dữ liệu JSON và truyền tin qua giao thức MQTT hoạt động hoàn hảo, không bị trễ hay mất gói tin.
  + **Phản ứng báo động và Đặc tính bão hòa ADC:** Tại kịch bản số 2 (kiểm thử rò rỉ), biểu đồ lịch sử trên Home Assistant ghi nhận 2 cột tín hiệu tăng vọt, tạo thành các chóp vuông chạm ngưỡng ~4000. Về mặt vật lý, mạch ESP32 sử dụng bộ chuyển đổi tín hiệu tương tự sang số (ADC) chuẩn 12-bit, tương đương dải đo từ 0 đến 4095. Khi gặp luồng khí Gas đậm đặc trực tiếp, điện áp trả về chân Analog bị bão hòa, đẩy thông số hiển thị lên mức cực đại. Điều này minh chứng cho độ nhạy cực cao của màng đo bên trong cảm biến. Khi ngừng xịt, biểu đồ lập tức sụt giảm về lại mức ~149, cho thấy cảm biến không bị "treo" mà phục hồi rất nhanh.
- **Đánh giá chung**: Phân hệ giám sát rò rỉ khí Gas đáp ứng xuất sắc các tiêu chuẩn an toàn kỹ thuật. Sự kết hợp giữa tốc độ đọc ADC tính bằng mili-giây của ESP32 và độ trễ truyền tải mạng cực thấp đã tạo ra một lớp bảo vệ vững chắc. Ngay khi luồng khí độc xuất hiện, thuật toán trên máy chủ lập tức ghi nhận và đồng bộ trạng thái (chuyển màu thẻ, cập nhật icon), đảm bảo người dùng nhận được cảnh báo kịp thời nhất để xử lý sự cố.

6.5. Hiển thị kết quả qua app Home assistant:

Giao diện người dùng (User Interface - UI) đóng vai trò là phương tiện giao tiếp trực tiếp giữa hệ thống và người quản lý. Trong nghiên cứu này, nền tảng Home Assistant được tùy biến sâu để không chỉ hiển thị các con số khô khan mà còn trực quan hóa dữ liệu, giúp người dùng dễ dàng theo dõi và đưa ra quyết định tức thời.



Hình 21: Giao diện giám sát chất lượng không khí tổng thể trên Home Assistant

Dựa trên kết quả triển khai thực tế, giao diện hiển thị được thiết kế và chia thành 3 phân hệ chức năng chính:

- **Phân hệ hiển thị thông số thời gian thực (Real-time Monitoring)**: Các dữ liệu thu thập từ nút mạng cảm biến (ESP32) được đẩy lên máy chủ qua giao thức MQTT và cập nhật liên tục trên giao diện với độ trễ gần như bằng 0. Hệ thống sử dụng các thẻ hiển thị (Cards) tối giản, hiện đại (ứng dụng thư viện Mushroom và Card-mod) để thể hiện các thông số đơn lẻ bao gồm: Nhiệt độ (30.4 °C), Độ ẩm (67.3%), Nồng độ eCO_2 (404 ppm) và Chỉ số TVOC (14 ppb). Các giá trị này đều đang nằm trong ngưỡng an toàn sinh hoạt.

  – **Phân hệ Đánh giá Chất lượng Không khí Tổng hợp (Dynamic Air Quality Card):** Đây là "trái tim" của giao diện cảnh báo. Thay vì bắt người dùng phải tự phân tích từng chỉ số, hệ thống sử dụng thuật toán chấm điểm (Scoring Algorithm) được lập trình bằng ngôn ngữ Jinja2 để tổng hợp dữ liệu từ 5 cảm biến.

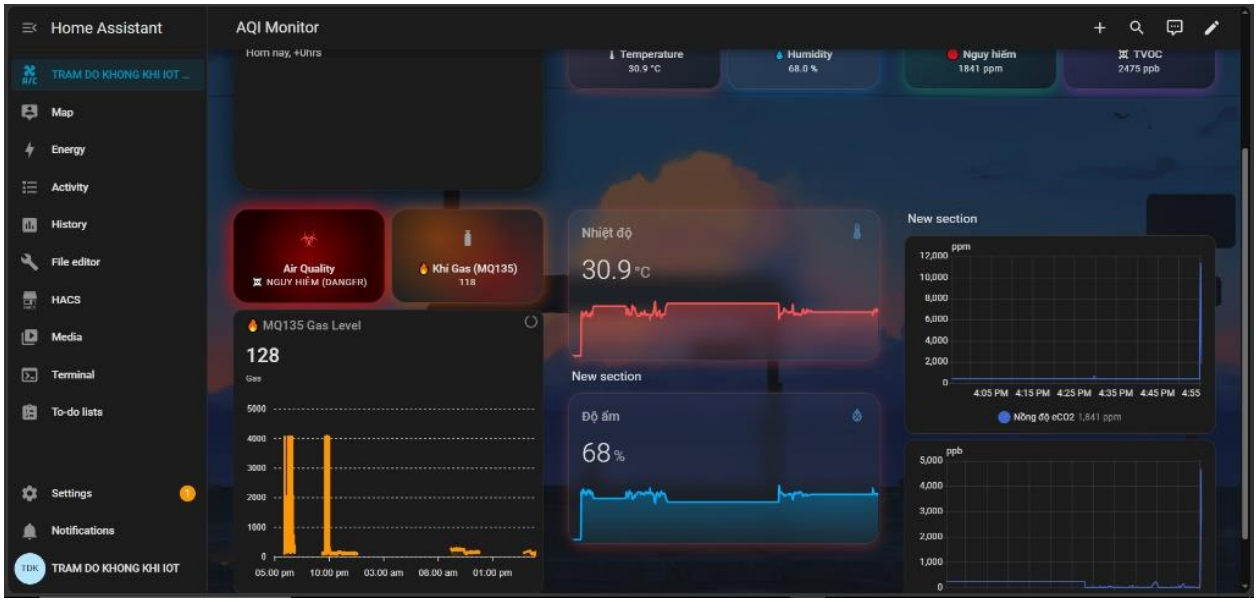
  – **Tính năng phản hồi thị giác (Visual Feedback):** Thẻ "Air Quality" có khả năng tự động thay đổi màu sắc và biểu tượng dựa trên mức độ nghiêm trọng. Tại thời điểm giám sát bình thường, thẻ hiển thị màu xanh lá với trạng thái "

RẤT TỐT (GOOD)". Nếu bất kỳ chỉ số nào (đặc biệt là khí độc Gas hoặc CO_2) vượt ngưỡng thiết lập, thẻ sẽ lập tức chuyển sang dải màu đỏ (Gradients) kèm hiệu ứng viền phát sáng (Box-shadow) và cảnh báo "☠ NGUY HIỂM", thu hút sự chú ý tuyệt đối của người quan sát.

– **Phân hệ Trực quan hóa Dữ liệu Lịch sử (Historical Data Visualization):** Giao diện tích hợp các biểu đồ dạng đường (Line charts) để lưu trữ và vẽ lại sự biến thiên của môi trường theo trục thời gian.

– **Đánh giá thực nghiệm qua biểu đồ:** Như quan sát tại biểu đồ "MQ135 Gas Level" (góc dưới bên trái), có thể thấy rõ các cột mốc tín hiệu vọt lên sát ngưỡng bão hòa của bộ ADC 12-bit (~4000). Đây là kết quả của quá trình thực nghiệm xịt khí Gas trực tiếp vào cảm biến để kiểm thử độ nhạy. Sau khi ngừng tác động, hệ thống ghi nhận chỉ số Gas nhanh chóng giảm dần và duy trì ổn định ở mức Baseline (khoảng 120-130), chứng minh khả năng phục hồi và đo lường chính xác của cảm biến. Tương tự, sự biến thiên của eCO_2 và TVOC cũng được ghi nhận chi tiết, phục vụ cho quá trình trích xuất dữ liệu và phân tích chuyên sâu sau này.

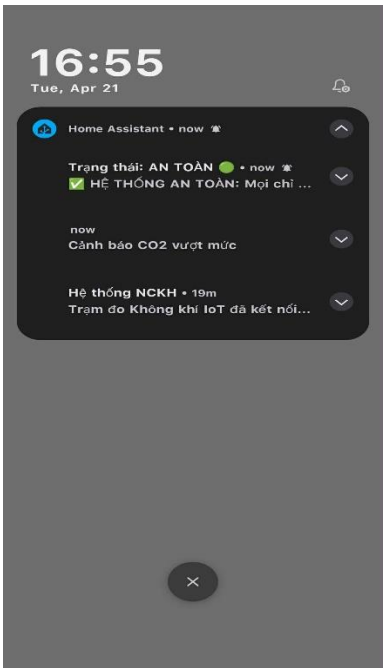
– **Thực nghiệm kích hoạt cảnh báo đồng bộ (Synchronized Alerting Experiment):** Để chứng minh tính hiệu quả của thuật toán cảnh báo, nhóm đã tiến hành kịch bản thực nghiệm khi chất lượng không khí suy giảm thực tế:



Hình 22: Giao diện cảnh báo đỏ khi eCO2 và TVOC vượt ngưỡng

+ **Đồng bộ trên giao diện Dashboard:** Khi nồng độ eCO2 tích tụ lên mức 1841 ppm và TVOC đạt 2475 ppb, thuật toán Jinja2 lập tức nhận diện mức độ rủi ro. Thẻ "Air Quality" tự động chuyển sang dải màu đỏ gắt kèm trạng

thái "☠ NGUY HIỂM (DANGER)", thay thế hoàn toàn màu xanh an toàn trước đó.



Hình 23: Chuỗi thông báo đẩy hiển thị trên màn hình khóa điện thoại di động

+ **Đồng bộ trên thiết bị di động:** song song với sự đổi màu trên Web, nền tảng tự động hóa đã gửi ngay lập tức thông báo khẩn ("Cảnh báo CO2 vượt mức") đến điện thoại người quản lý. Đặc biệt, sau khi không gian được thông gió, hệ thống tiếp tục gửi bản tin xác nhận ("Trạng thái: AN TOÀN 🟢 - HỆ THỐNG AN TOÀN..."), chứng minh cơ chế độ trễ và kiểm tra chéo (And if) hoạt động hoàn hảo, không gây nhiễu thông báo cho người dùng.

  ⇒ **Nhận xét:** Giao diện hiển thị được thiết kế với tông màu tối (Dark theme / Glassmorphism) không chỉ mang lại tính thẩm mỹ cao mà còn giúp làm nổi bật các cảnh báo khẩn cấp. Nền tảng Home Assistant cho phép truy cập đồng bộ trên cả trình duyệt Web (PC) và Ứng dụng di động (Mobile App), đảm bảo người dùng có thể giám sát chất lượng không khí tại phòng mọi lúc, mọi nơi.

TÀI LIỆU THAM KHẢO

[1] L. Miasayedava, J. A. Tuhtan, and J. Kaugerand, "Lightweight Open Data Assimilation of Pan-European Urban Air Quality," *IEEE Access*, 2023. [Online]. Available: https://www.mdpi.com/2076-3417/14/13/5774

[2] I. D. Apostolopoulos, E. Dovrou, *et al.*, "Monitoring of Indoor Air Quality in a Classroom Combining a Low-Cost Sensor System and Machine Learning," *Chemosensors*, vol. 13, no. 4, p. 148, 2025. [Online]. Available: https://doi.org/10.3390/chemosensors13040148

[3] R. G. Barrientos-Mauricio, "IoT-based air pollution monitoring system: A case study in Artisanal Brick Kilns," *Sustainable Environment*, 2025. [Online]. Available: https://doi.org/10.1080/27658511.2025.2607841

[4] "Nghiên cứu thiết kế hệ thống giám sát không khí," *DLU Journal of Science*, 2023. [Online]. Available: https://scholar.dlu.edu.vn/thuvienso/bitstream/DLU123456789/268283/1/77140-937-182746-1-10-20230320.pdf

[5] "Hệ thống giám sát chất lượng không khí," *JST-UD*, 2023. [Online]. Available: https://jst-ud.vn/jst-ud/article/view/8023/5717

[6] J. Saini, M. Dutta, and G. Marques, "Indoor Air Quality Monitoring Systems Based on Internet of Things: A Systematic Review," *International Journal of Environmental Research and Public Health*, vol. 17, no. 14, p. 4942, 2020. [Online]. Available: https://www.mdpi.com/1660-4601/17/14/4942

[7] A. Zivelonghi and A. Giuseppi, "Smart Healthy Schools: An IoT-enabled concept for multi-room dynamic air quality control," *Internet of Things and Cyber-Physical Systems*, vol. 4, pp. 24-31, 2024. [Online]. Available: https://doi.org/10.1016/j.iotcps.2023.05.001

[8] N. Salman, "Real Time Wireless Sensor Network (WSN) based Indoor Air Quality Monitoring System," *IFAC-PapersOnLine*, vol. 52, no. 24, pp. 334-339, 2019. [Online]. Available: https://doi.org/10.1016/j.ifacol.2019.12.430

[9] "Designing a low-cost wireless sensor network for Air Quality Monitoring," *White Rose Research Online*. [Online]. Available: https://link.springer.com/article/10.1007/s10776-016-0299-y

[10] "Wireless Sensing Network for Implementation of Air Quality Monitoring System in University of Baghdad," *ScienceDirect*. [Online]. Available: https://doi.org/10.46717/igj.2D.57.17ms-2024-10-27

[11] "SchoolAIR: A Citizen Science IoT Framework Using Low-Cost Sensors," *MDPI*. [Online]. Available: https://doi.org/10.3390/s24010148

[12] M. Schilling *et al.*, "Air quality monitoring survey in german school classrooms during the COVID-19 pandemic 2021," *arXiv preprint arXiv:2203.13500*, 2022. [Online]. Available: https://arxiv.org/abs/2203.13500

[13] "Improving classroom air quality with IoT-driven CO_2 display (SAVE unit)," *ScienceDirect*. [Online]. Available: https://doi.org/10.1016/j.scitotenv.2025.179543

PHỤ LỤC

CODE Arduino IDE:

```
#include <Wire.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include "Adafruit_SGP30.h"
#include "DHT.h"
const char* ssid = "Tên wifi máy chủ kết nối";
const char* password = " Mật khẩu wifi máy chủ kết nối ";
const char* mqtt_server = "ip từ Ipv4 trong home assistant";
const char* mqtt_user = "esp32";
const char* mqtt_pass = "123456";
#define BUZZER_PIN 15
#define ECO2_THRESHOLD 2000
#define MQ135_PIN 33
int MQ135_THRESHOLD = 0;
#define DHTPIN 14
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
#define HUMIDITY_THRESHOLD 85
Adafruit_SGP30 sgp;
WiFiClient espClient;
PubSubClient mqttClient(espClient);
long lastSendTime = 0;
void setup_wifi() {
  Serial.print("\nĐang ket noi WiFi: ");
  Serial.println(ssid);
```

```
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi DA KET NOI!");
}
void reconnect() {
  while (!mqttClient.connected()) {
    Serial.print("Đang ket noi Home Assistant...");
    String clientId = "NCKH_TRAM_" + String(random(0xffff), HEX);
    if (mqttClient.connect(clientId.c_str(), mqtt_user, mqtt_pass)) {
      Serial.println(" THANH CONG!");
      mqttClient.publish("nckh/trang_thai", "ONLINE");
    } else {
      Serial.print(" THAT BAI, thu lai sau 5s...");
      delay(5000);
    }
  }
}
void setup() {
  Serial.begin(115200);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, HIGH);
  dht.begin();
  setup_wifi();
  mqttClient.setServer(mqtt_server, 1883);
  if (!sgp.begin()){
    Serial.println("LOI: Khong tim thay SGP30!");
```

```
  while (1);
 }
 Serial.println("==================================");
 Serial.println("Bat dau hieu chuan MQ135 (5 giay)....");
 long tong_gia_tri = 0;
 for (int i = 0; i < 50; i++) {
  tong_gia_tri += analogRead(MQ135_PIN);
  delay(100);
 }
 int gia_tri_nen = tong_gia_tri / 50;
 MQ135_THRESHOLD = gia_tri_nen + 800;
 Serial.print("-> Gia tri khong khi nen: ");
 Serial.println(gia_tri_nen);
 Serial.print("-> NGUONG BAO DONG DA CHOT: ");
 Serial.println(MQ135_THRESHOLD);
 Serial.println("Hieu chuan hoan tat. He thong bat dau chay!");
 Serial.println("==================================");
 digitalWrite(BUZZER_PIN, LOW);
 delay(200);
 digitalWrite(BUZZER_PIN, HIGH);
 tone(BUZZER_PIN, 2000); delay(200);
 noTone(BUZZER_PIN); digitalWrite(BUZZER_PIN, HIGH);
 Serial.println("Cảm biến đang làm nóng (15 giây)... Vui lòng đợi!");
 for (int i = 15; i > 0; i--) {
  Serial.print(i); Serial.print("s... ");
  delay(1000);
 }
 Serial.println("\n>> HỆ THỐNG ĐÃ SẴN SÀNG ĐO LƯỜNG! <<\n");
}
```

```
void loop() {
  if (!mqttClient.connected()) reconnect();
  mqttClient.loop();
  if (!sgp.IAQmeasure()) return;
  int mq135_value = analogRead(MQ135_PIN);
  float h = dht.readHumidity();
  if (isnan(h) || isnan(t)) {
    Serial.println("Lỗi: Không đọc được dữ liệu từ DHT11!");
    h = 0.0;
    t = 0.0;
  }
  if (sgp.eCO2 > ECO2_THRESHOLD ||
     (mq135_value >= MQ135_THRESHOLD && mq135_value >= 2000) ||
      h > HUMIDITY_THRESHOLD)
  {
      tone(BUZZER_PIN, 1500); delay(200);
      noTone(BUZZER_PIN); digitalWrite(BUZZER_PIN, LOW); delay(100);
      tone(BUZZER_PIN, 1000); delay(200);
      noTone(BUZZER_PIN); digitalWrite(BUZZER_PIN, HIGH); delay(100);
  }
  else {
      noTone(BUZZER_PIN);
      digitalWrite(BUZZER_PIN, HIGH);
  }
  long now = millis();
  if (now - lastSendTime > 5000) {
    lastSendTime = now;
    Serial.println("--- THÔNG SỐ MÔI TRƯỜNG ---");
    Serial.print("Chi so hien tai -> Nhiệt độ: "); Serial.print(t); Serial.print("°C | ");
```

```
    Serial.print("Độ ẩm: "); Serial.print(h); Serial.print("% | ");
    Serial.print("eCO2: ");Serial.print(sgp.eCO2);
    Serial.print(" ppm  |   TVOC: ");Serial.print(sgp.TVOC);
    Serial.print(" ppb  |   MQ135: ");Serial.println(mq135_value);
    String payload = "{";
    payload += "\"Temperature\":" + String(t) + ",";
    payload += "\"Humidity\":" + String(h) + ",";
    payload += "\"eCO2\":" + String(sgp.eCO2) + ",";
    payload += "\"TVOC\":" + String(sgp.TVOC) + ",";
    payload += "\"MQ135\":" + String(mq135_value);
    payload += "}";
    bool guiThanhCong = mqttClient.publish("nckh khong khi IOT", payload.c_str());
  }
}
```

CODE khai báo cảm biến MQTT:

```
default_config:
mobile_app:
automation: !include automations.yaml
script: !include scripts.yaml
scene: !include scenes.yaml
mqtt:
  sensor:
    - name: "Nhiệt độ phòng"
      state_topic: "nckh khong khi IOT"
      value_template: "{{ value_json.Temperature }}"
      unit_of_measurement: "°C"
      device_class: temperature
    - name: "Độ ẩm phòng"
      state_topic: "nckh khong khi IOT"
```

```
  value_template: "{{ value_json.Humidity }}"
  unit_of_measurement: "%"
  device_class: humidity
- name: "Nồng độ eCO2"
  state_topic: "nckh khong khi IOT"
  value_template: "{{ value_json.eCO2 }}"
  unit_of_measurement: "ppm"
  icon: mdi:molecule-co2
- name: "Chỉ số Bụi/Khí độc TVOC"
  state_topic: "nckh khong khi IOT"
  value_template: "{{ value_json.TVOC }}"
  unit_of_measurement: "ppb"
  icon: mdi:air-filter
- name: "Chỉ số Rò rỉ Gas (MQ135)"
  state_topic: "nckh khong khi IOT"
  value_template: "{{ value_json.MQ135 }}"
  icon: mdi:gas-cylinder
  state_class: measurement
```